Chương 9: Iterator Pattern và Composite Pattern (P1)



Iterator Pattern và Composite Pattern (https://toihocdesignpattern.com/chuong-9-iterator-pattern-va-composite-pattern-p2-composite-pattern.html) – Quản lý bộ sưu tập

Có rất nhiều cách để nhét object vào một collection. Đặt chúng trong một **Array,** một **Stack**, một **List**, một **Hashtable**, tùy lựa chọn. Mỗi loại có những lợi thế và sự đánh đổi riêng. Nhưng đến một lúc nào đó, client của bạn sẽ muốn duyệt qua những đối tượng đó, và khi đó, bạn sẽ tự tay cài đặt thuật toán duyệt của mình (duyệt 1 mảng sẽ khác duyệt 1 **ArrayList**), bạn có muốn như vậy? Chúng tôi hy vọng không! Điều đó không chuyên nghiệp. Khi xem qua phần **Iterator pattern**, bạn sẽ thấy cách bạn có thể cho phép client của mình duyệt qua các đối tượng mà không cần phải xem qua cách bạn lưu trữ các đối tượng. Bạn cũng sẽ học cách tạo ra một số bộ super collection có thể bỏ qua một số cấu trúc dữ liệu trong một ràng buộc duy nhất. Và nếu điều đó chưa đủ, bạn cũng sẽ học được một hoặc hai thứ về **trách nhiệm của đối tượng**.

Tin tức mới nhất: Objectville Diner và Objectville Pancake sáp nhập

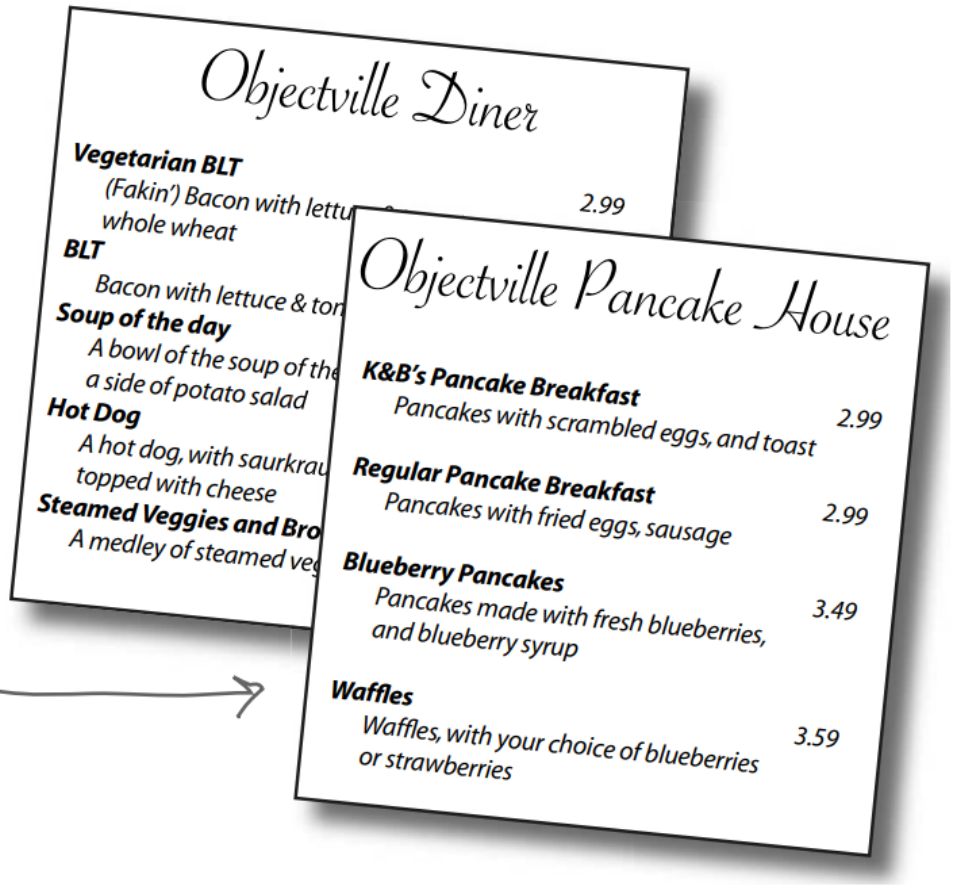
Tin tốt đấy! Bây giờ chúng ta có thể có được những bữa sáng và những bữa trưa ngon tuyệt ở một cùng nơi. Không phải sáng ở Objectville Pancake và trưa lại ở Objectville Diner nữa. Nhưng dường như có một vấn đề nhỏ…



(Trong đoạn trao đổi: Mẹ dùng Mảng và Lou ArrayList để lưu menu)

Kiểm tra qua các menu

Ít nhất là Lou và Mel đồng ý về việc triển khai **MenuItems**. Hãy cùng kiểm tra các mục trên mỗi menu, và hãy xem cách thực hiện.



Diner menu có rất nhiều món ăn trưa, trong khi Pancake House bao gồm các món ăn sáng. Mỗi mục menu có tên, mô tả và giá cả

```
public class MenuItem {
    String name;
    String description;
    boolean vegetarian;
    double price;

    public MenuItem(String name,
                    String description,
                    boolean vegetarian,
                    double price)
    {
        this.name = name;
        this.description = description;
        this.vegetarian = vegetarian;
        this.price = price;
    }

    public String getName() {
        return name;
    }

    public String getDescription() {
        return description;
    }

    public double getPrice() {
        return price;
    }

    public boolean isVegetarian() {
        return vegetarian;
    }
}
```

MenuItem bao gồm tên, mô tả, cờ để cho món đó có phải là món chay hay không và giá. Bạn chuyển tất cả các giá trị này vào hàm tạo để khởi tạo MenuItem.

Các phương thức getter này cho phép bạn truy cập vào các trường của menu item.

Cài đặt menu của Lou và Mel

Bây giờ, hãy nhìn về những gì Lou và Mel đang tranh cãi. Cả hai đều tốn rất nhiều thời gian và code được đầu tư theo cách họ lưu trữ các item trong một menu, và rất nhiều code khác phụ thuộc vào nó.

Lou: Tôi đã sử dụng một ArrayList để tôi có thể dễ dàng thêm mới một menu item.

```
public class PancakeHouseMenu {
    ArrayList menuItems;

    public PancakeHouseMenu() {
        menuItems = new ArrayList();

        addItem("K&B's Pancake Breakfast",
            "Pancakes with scrambled eggs, and toast",
            true,
            2.99);

        addItem("Regular Pancake Breakfast",
            "Pancakes with fried eggs, sausage",
            false,
            2.99);

        addItem("Blueberry Pancakes",
            "Pancakes made with fresh blueberries",
            true,
            3.49);

        addItem("Waffles",
            "Waffles, with your choice of blueberries or strawberries",
            true,
            3.59);
    }
    public void addItem(String name, String description,
                        boolean vegetarian, double price)
    {
        MenuItem menuItem = new MenuItem(name, description, vegetarian, price);
        menuItems.add(menuItem);
    }

    public ArrayList getMenuItems() {
        return menuItems;
    }

    // other menu methods here
}
```

Lou sử dụng một ArrayList để lưu trữ các menu item của mình

Mỗi menu item được thêm vào ArrayList ở đây, trong hàm khởi tạo

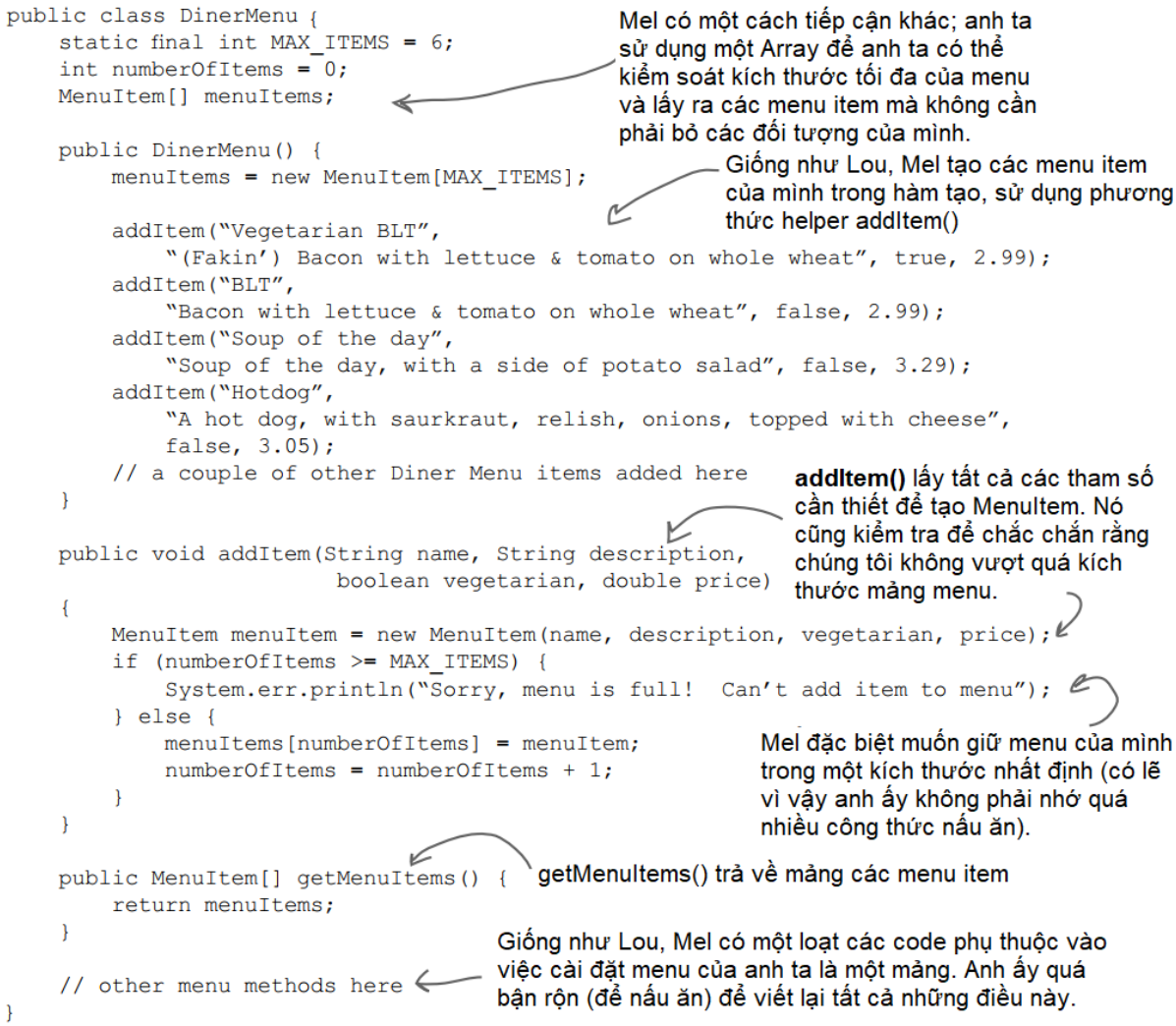
Mỗi MenuItem có một tên, một mô tả, là món chay hay không (true, false) và giá cả

Để thêm một menu item, Lou tạo một đối tượng MenuItem mới, truyền vào đối số và sau đó thêm nó vào ArrayList

Phương thức getMenuItems() trả về danh sách các menu item

Lou có một loạt các đoạn code menu khác phụ thuộc vào việc triển khai ArrayList. Anh ta không muốn phải viết lại tất cả code đó!

Mel: Haaa! Một Arraylist … Tôi đã sử dụng một array THỰC SỰ để tôi có thể kiểm soát kích thước tối đa của menu và get MenuItems của mình mà không phải sử dụng cast.

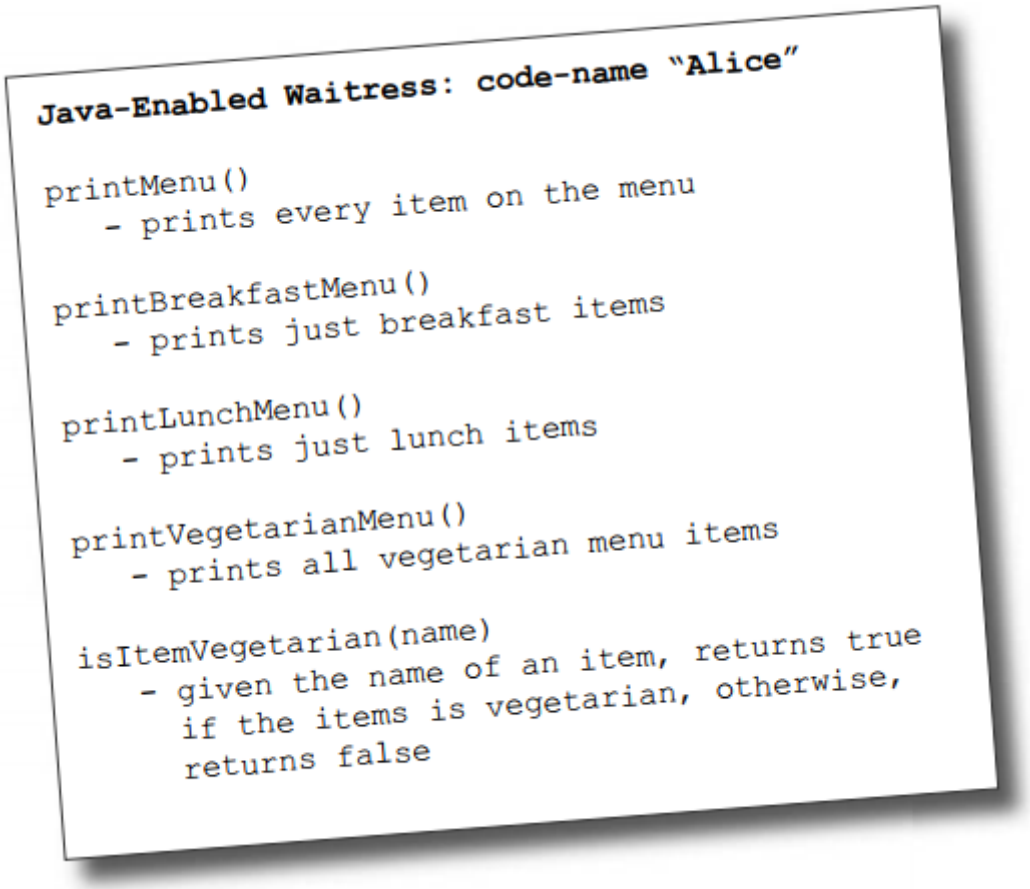


Vấn đề với việc có hai biểu diễn menu khác nhau là gì?

Để xem lý do tại sao việc có hai biểu diễn menu khác nhau lại làm phức tạp mọi thứ, hãy thử thực hiện một ứng dụng client sử dụng hai menu. Hãy tưởng tượng bạn đã được thuê bởi công ty mới được thành lập bởi sự hợp nhất của Diner và Pancake House để tạo ra một phục vụ **Java-enabled Waitress**. Mô tả cho người phục vụ để cô ấy có thể in một menu theo khách hàng theo yêu cầu và thậm chí cho bạn biết khi một menu item là món chay mà không cần phải hỏi đầu bếp – bây giờ là một sự đổi mới!

Hãy kiểm tra mô tả, và sau đó bước qua những gì có thể cần để thực hiện…

Mô tả cho Java-Enabled Waitress (client)



Hãy bắt đầu bằng cách xem qua cách chúng tôi thực hiện phương thức **printMenu()**:

1. Để in tất cả các item trên mỗi menu, bạn sẽ cần gọi phương thức **getMenuItem()** trên **PancakeHouseMenu** và **DinerMenu** để lấy các menu item tương ứng của chúng. Lưu ý rằng mỗi phương thức trả về một kiểu lưu trữ khác nhau (mảng và ArrayList):

```
PancakeHouseMenu pancakeHouseMenu = new PancakeHouseMenu();
ArrayList breakfastItems = pancakeHouseMenu.getMenuItems();

DinerMenu dinerMenu = new DinerMenu();
MenuItem[] lunchItems = dinerMenu.getMenuItems();
```

2 phương thức này trông giống nhau, nhưng các cuộc gọi đang trả về các loại dữ liệu khác nhau

Các menu ăn sáng nằm trong một **ArrayList**, các menu ăn trưa nằm trong một **Array**.

2. Bây giờ, để in ra các item từ **PancakeHouseMenu**, chúng tôi sẽ duyệt qua các item trên ArrayList **breakfastItems**. Và để in ra các Diner item, chúng tôi sẽ duyệt qua một mảng.

```
for (int i = 0; i < breakfastItems.size(); i++) {
    MenuItem menuItem = (MenuItem)breakfastItems.get(i);
    System.out.print(menuItem.getName() + " ");
    System.out.println(menuItem.getPrice() + " ");
    System.out.println(menuItem.getDescription());
}

for (int i = 0; i < lunchItems.length; i++) {
    MenuItem menuItem = lunchItems[i];
    System.out.print(menuItem.getName() + " ");
    System.out.println(menuItem.getPrice() + " ");
    System.out.println(menuItem.getDescription());
}
```

Bây giờ, chúng ta phải thực hiện hai vòng lặp khác nhau để duyệt qua các menu item...

.. một vòng lặp cho ArrayList ...

và một cái khác cho Mảng.

Thực hiện mọi phương thức khác trong Waitress sẽ là một biến thể của chương này. Chúng tôi luôn luôn cần có cả hai menu và sử dụng hai vòng để lặp qua các item của chúng. Nếu một nhà hàng khác có triển khai khác được gộp thì chúng tôi sẽ có ba vòng lặp.

Bài tập:

Dựa trên việc triển khai **printMenu()** của chúng tôi, áp dụng nào sau đây?

- ❑ A. Chúng tôi đang code cho các triển khai cụ thể của PancakeHouseMenu và DinerMenu, chứ không phải cho một interface.
- ❑ B. Waitress không implement Java Waitress API và vì vậy cô ấy không tuân thủ theo tiêu chuẩn.
- ❑ C. Nếu chúng tôi quyết định chuyển từ sử dụng DinerMenu sang một loại menu khác đã triển khai danh sách các menu item của nó bằng Hashtable, chúng tôi đã phải sửa đổi rất nhiều code trong Waitress.
- ❑ D. Waitress cần biết làm thế nào mỗi menu đại diện cho tập hợp các menu item bên trong của nó; Điều này vi phạm nguyên tắc đóng gói.
- ❑ E. Chúng tôi có code trùng lặp: phương thức printMenu() cần hai vòng lặp riêng biệt để lặp qua hai loại menu khác nhau. Và nếu chúng ta thêm một menu thứ ba, chúng ta sẽ có một vòng lặp khác.
- ❑ F. Việc triển khai không dựa trên MXML (Menu XML) và do đó không thể tương thích như mong muốn.

Đáp án: A, C, D, E

Bây giờ thì sao?

Mel và Lou đang đặt chúng ta vào thế khó. Họ không muốn thay đổi cách triển khai vì điều đó có nghĩa là viết lại rất nhiều code trong mỗi lớp menu tương ứng. Nhưng nếu một trong số họ không nhượng bộ, thì chúng tôi sẽ có công việc của Waitress sẽ khó duy trì và mở rộng.

Sẽ thật tuyệt nếu chúng ta có thể tìm cách cho phép chúng implements cùng một interface cho các menu. Hiện tại chúng đã đóng (không có implement), ngoại trừ kiểu trả về của phương thức **getMenuItems()**. Bằng cách đó, chúng ta có thể giảm thiểu các tham chiếu cụ thể trong code Waitress và cũng hy vọng thoát khỏi nhiều vòng lặp cần thiết để duyệt trên cả hai menu.

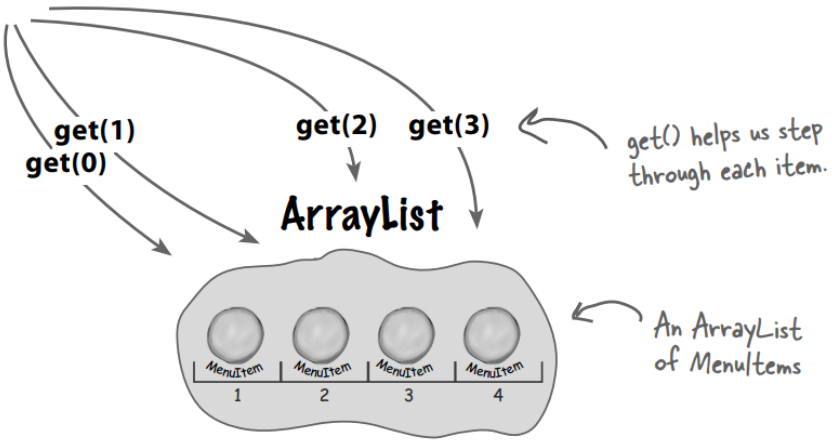
Nghe hay đấy? Chà, làm thế nào chúng ta làm được điều đó?

Chúng ta có thể gói gọn việc duyệt phần tử không?

Nếu chúng ta đã được học một điều từ cuốn sách này, đó là đóng gói những gì thay đổi. Những gì đang thay đổi ở đây đã rõ ràng: "duyệt phần tử", gây ra bởi các kiểu tập hợp khác nhau, được return từ các menu. Nhưng chúng ta có thể đóng gói điều này không? Hãy suy nghĩ một chút…

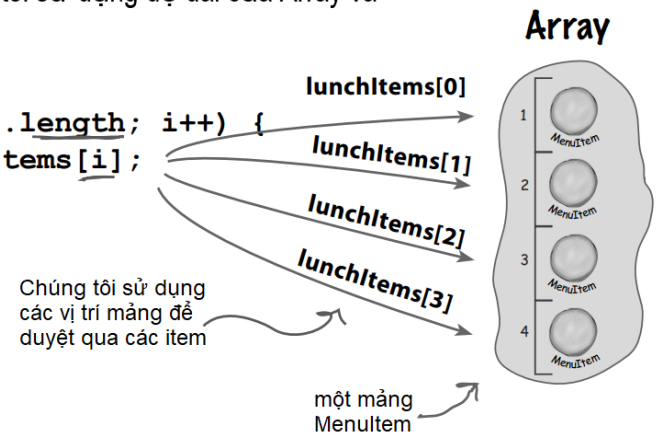
❶ Để lặp qua các breakfast item, chúng tôi sử dụng các phương thức size() và get() trên ArrayList:

```
for (int i = 0; i < breakfastItems.size(); i++) {
    MenuItem menuItem = (MenuItem)breakfastItems.get(i);
}
```



❷ Và để lặp lại qua các lunch item, chúng tôi sử dụng độ dài của Array và index trên Mảng MenuItem.

```
for (int i = 0; i < lunchItems.length; i++) {
    MenuItem menuItem = lunchItems[i];
}
```



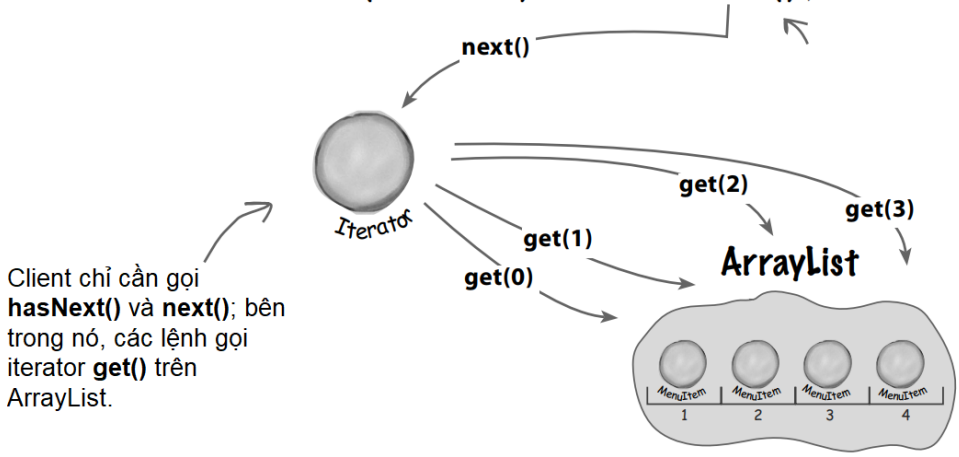
❸ Bây giờ nếu chúng ta tạo một đối tượng, hãy gọi nó là Iterator, nó gói gọn cách chúng ta lặp qua một bộ tập hợp các đối tượng? Hãy để thử cái này trên ArrayList

Chúng tôi yêu cầu breakfastMenu tạo một iterator của MenuItem của nó

```
Iterator iterator = breakfastMenu.createIterator();

while (iterator.hasNext()) {
    MenuItem menuItem = (MenuItem)iterator.next();
}
```

Và trong khi chưa duyệt qua hết item...



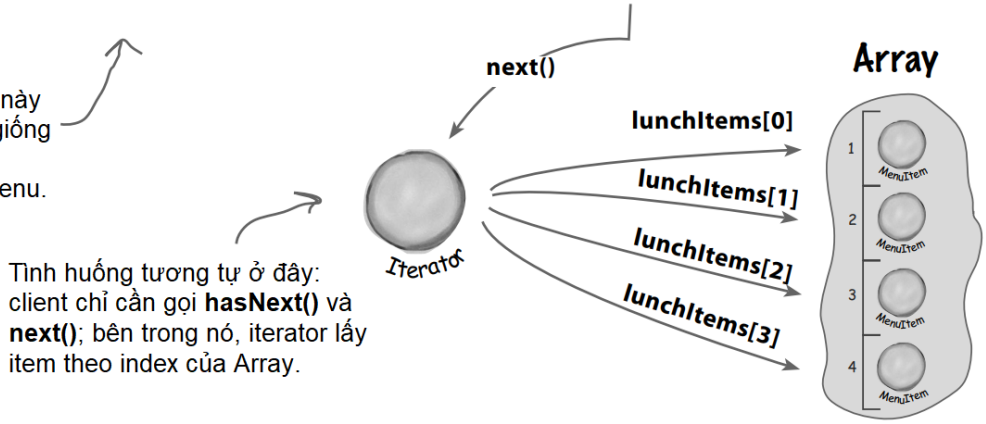
Client chỉ cần gọi **hasNext()** và **next()**; bên trong nó, các lệnh gọi iterator **get()** trên ArrayList.

❹ Hãy cùng thử với điều đó trên Array:

```
Iterator iterator = lunchMenu.createIterator();

while (iterator.hasNext()) {
    MenuItem menuItem = (MenuItem)iterator.next();
}
```

Đoạn code này hoàn toàn giống với code BreakfastMenu.

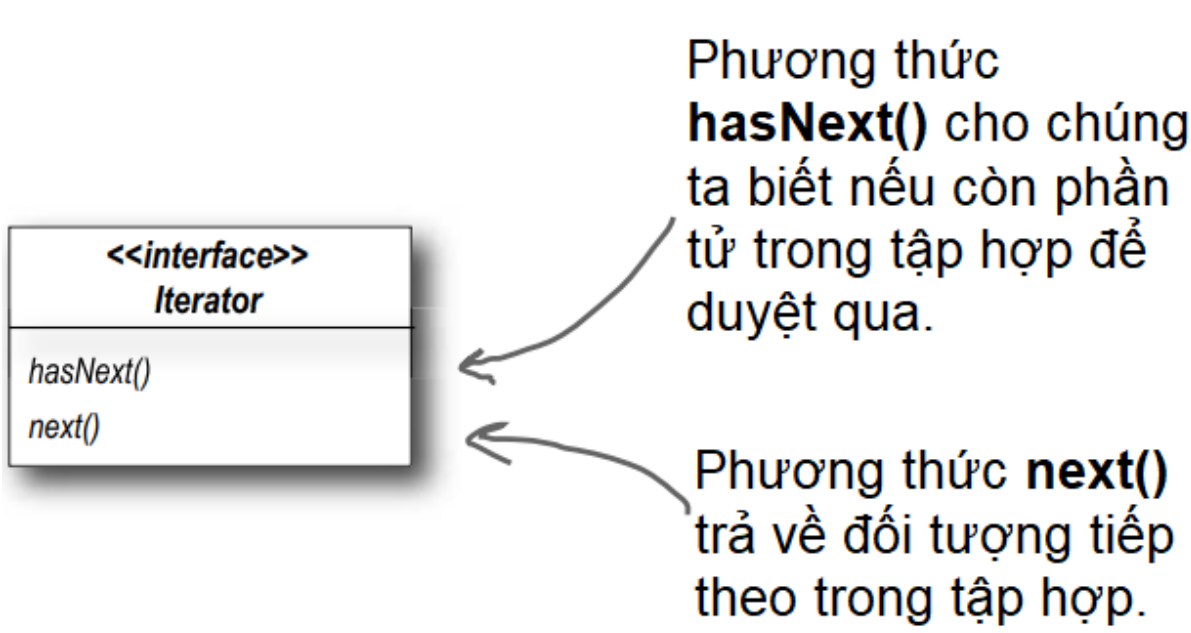


Tình huống tương tự ở đây: client chỉ cần gọi **hasNext()** và **next()**; bên trong nó, iterator lấy item theo index của Array.

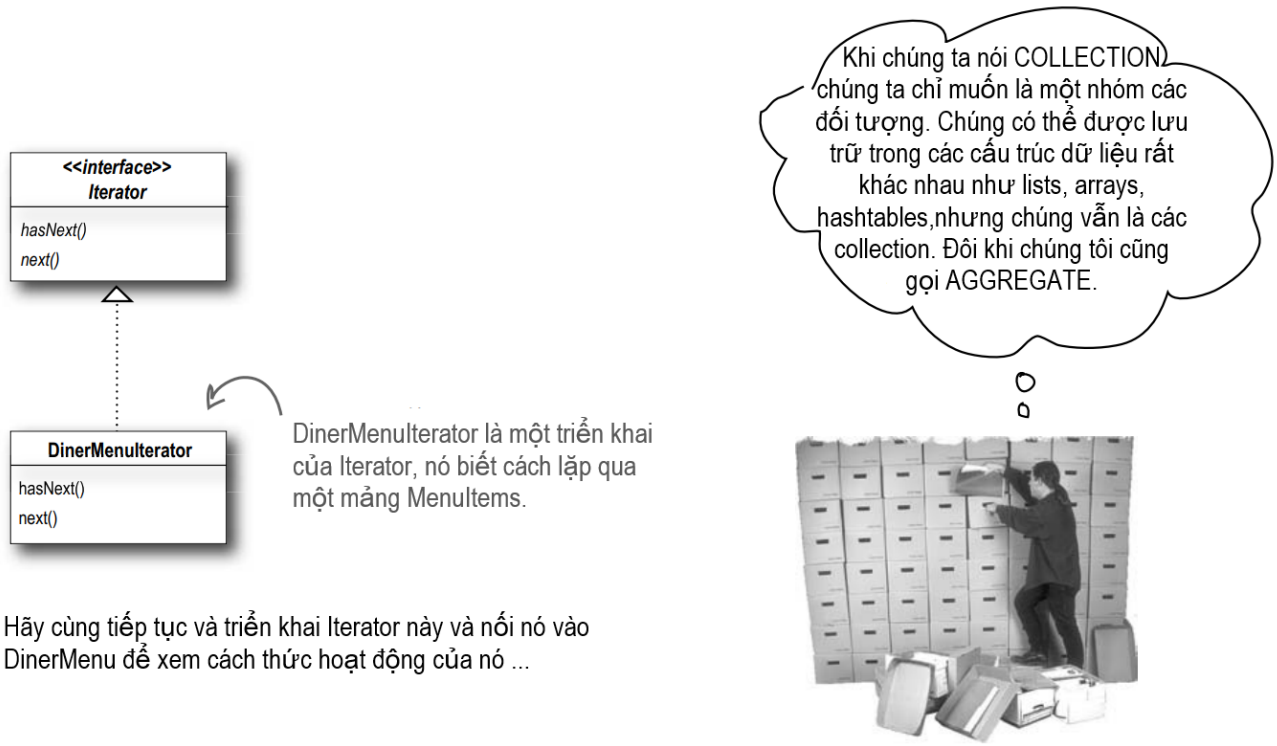
Gặp gỡ Iterator Pattern

Chà, có vẻ như kế hoạch đóng gói của chúng ta thực sự có thể hoạt động; và bạn có thể đã biết từ đầu tiêu đề, Mẫu thiết kế này được gọi là **Iterator Pattern**.

Điều đầu tiên bạn cần biết về **Iterator Pattern** là nó dựa trên interface có tên là Iterator. Ở đây, một Iterator interface có thể có:

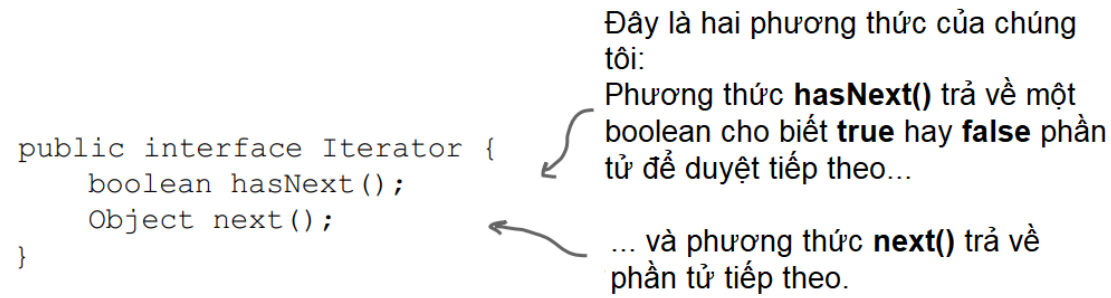


Bây giờ, khi chúng ta có giao diện này, chúng ta có thể triển khai **Iterator pattern** cho bất kỳ loại tập hợp đối tượng nào: mảng, danh sách, hashtables, … chọn loại tập hợp đối tượng yêu thích của bạn. Khi chúng ta muốn triển khai **Iterator pattern** cho Array được sử dụng trong **DinerMenu**. Nó sẽ trông như thế này:



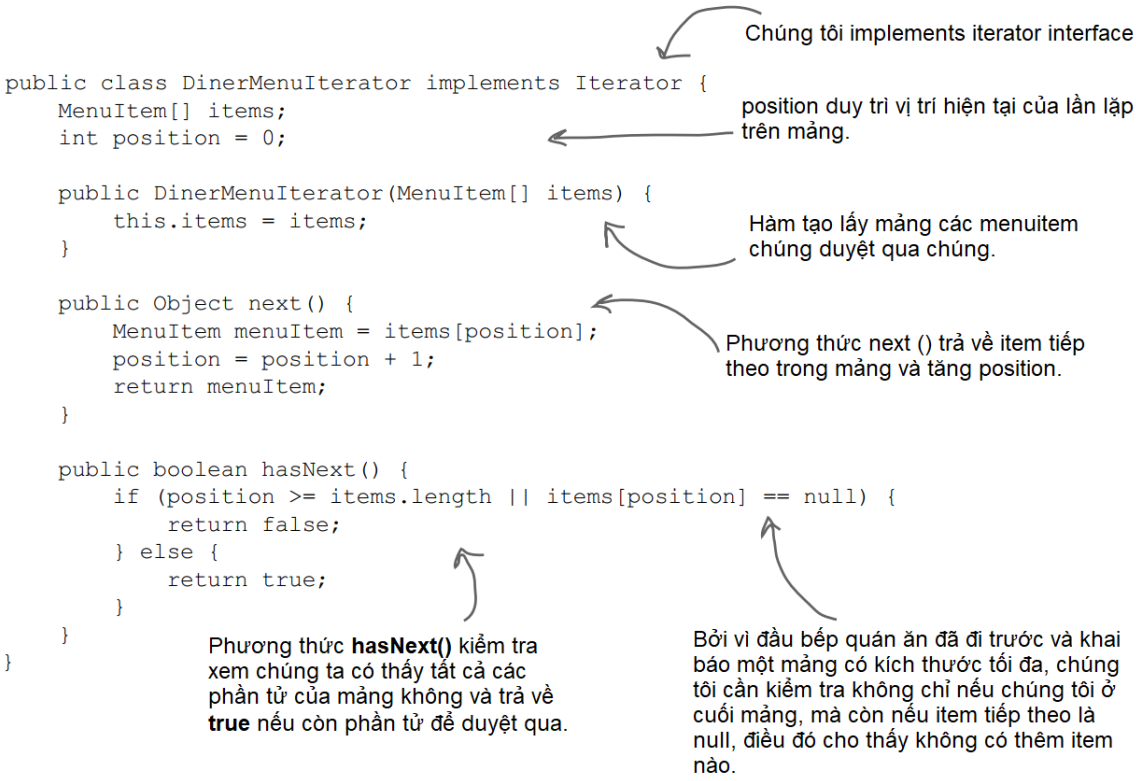
Áp dụng Iterator Pattern vào DinerMenu

Để thêm **Iterator** vào **DinerMenu**, trước tiên chúng ta cần định nghĩa **Iterator** Interface:




```
public interface Iterator {
    boolean hasNext();
    Object next();
}
```

Và bây giờ chúng ta cần triển khai một Iterator con đại diện cho Diner Menu:




```
public class DinerMenuIterator implements Iterator {
    MenuItem[] items;
    int position = 0;

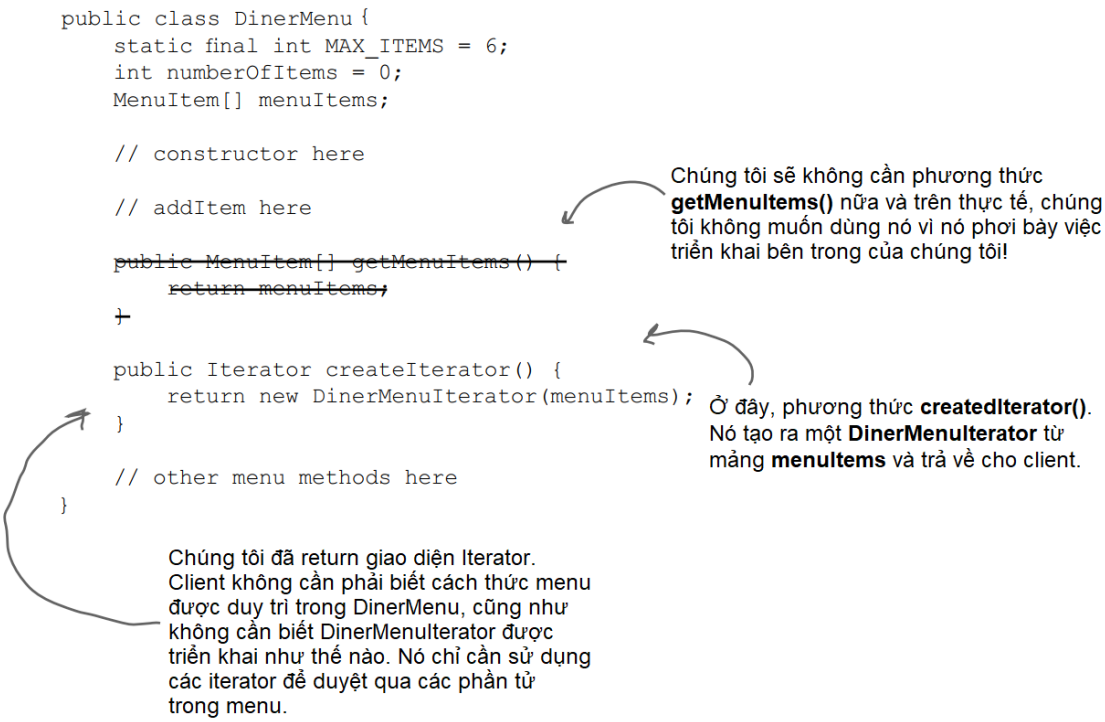
    public DinerMenuIterator(MenuItem[] items) {
        this.items = items;
    }

    public Object next() {
        MenuItem menuItem = items[position];
        position = position + 1;
        return menuItem;
    }

    public boolean hasNext() {
        if (position >= items.length || items[position] == null) {
            return false;
        } else {
            return true;
        }
    }
}
```

Làm lại DinerMenu với Iterator Pattern

Được rồi, chúng tôi đã có các iterator. Thời gian để làm việc với **DinerMenu**; tất cả những gì chúng ta cần làm là thêm một phương thức để tạo **DinerMenuIterator** và return lại cho client:



Bài tập:

Hãy tiếp tục và tự mình thực hiện **PancakeHouseIterator** và thực hiện các thay đổi cần thiết để kết hợp nó vào **PancakeHouseMenu**.

SỬA CODE WAITRESS

Bây giờ chúng ta cần tích hợp **iterator pattern** vào Waitress. Chúng ta sẽ có thể loại bỏ một số dư thừa trong tiến trình. Việc tích hợp khá đơn giản: đầu tiên chúng ta tạo một phương thức **printMenu()**, sau đó chúng ta sử dụng phương thức **createIterator()** trên mỗi menu để lấy Iterator và chuyển nó sang phương thức mới.

```
public class Waitress {
    PancakeHouseMenu pancakeHouseMenu;
    DinerMenu dinerMenu;

    public Waitress(PancakeHouseMenu pancakeHouseMenu, DinerMenu dinerMenu) {
        this.pancakeHouseMenu = pancakeHouseMenu;
        this.dinerMenu = dinerMenu;
    }

    public void printMenu() {
        Iterator pancakeIterator = pancakeHouseMenu.createIterator();
        Iterator dinerIterator = dinerMenu.createIterator();
        System.out.println("MENU\n----\nBREAKFAST");
        printMenu(pancakeIterator);
        System.out.println("\nLUNCH");
        printMenu(dinerIterator);
    }

    private void printMenu(Iterator iterator) {
        while (iterator.hasNext()) {
            MenuItem menuItem = (MenuItem)iterator.next();
            System.out.print(menuItem.getName() + ", ");
            System.out.print(menuItem.getPrice() + " -- ");
            System.out.println(menuItem.getDescription());
        }
    }

    // other methods here
}
```

Trong hàm khởi tạo, Waitress có hai menu.

Phương thức **printMenu()** bây giờ tạo hai iterator, một cho mỗi menu.

Và sau đó gọi **printMenu()** được overload với mỗi iterator.

Kiểm tra nếu iterator còn phần tử để duyệt

lấy phần tử tiếp theo

Phương thức **printMenu()** được overload sử dụng Iterator để duyệt qua các menu item và in chúng.

Sử dụng các **menuItem** để có được tên, giá, mô tả và in chúng

Lưu ý rằng: chúng tôi đã giảm bớt một vòng lặp

TESTING CODE CỦA BẠN

Đây là thời gian để đưa mọi thứ vào một thử nghiệm. Hãy viết một số test drive và xem cách Waitress hoạt động…

```
public class MenuTestDrive {
    public static void main(String args[]) {
        PancakeHouseMenu pancakeHouseMenu = new PancakeHouseMenu();
        DinerMenu dinerMenu = new DinerMenu();

        Waitress waitress = new Waitress(pancakeHouseMenu, dinerMenu);

        waitress.printMenu();
    }
}
```

Đầu tiên chúng ta tạo các menu mới.

Sau đó, chúng tôi tạo ra một Waitress và chuyển cho cô ấy các menu.

Khi đó chúng tôi in chúng.

Đây là kết quả…

```
File Edit Window Help GreenEggs&Ham
% java DinerMenuTestDrive

MENU
----
BREAKFAST
K&B's Pancake Breakfast, 2.99 -- Pancakes with scrambled eggs, and toast
Regular Pancake Breakfast, 2.99 -- Pancakes with fried eggs, sausage
Blueberry Pancakes, 3.49 -- Pancakes made with fresh blueberries
Waffles, 3.59 -- Waffles, with your choice of blueberries or strawberries

LUNCH
Vegetarian BLT, 2.99 -- (Fakin') Bacon with lettuce & tomato on whole wheat
BLT, 2.99 -- Bacon with lettuce & tomato on whole wheat
Soup of the day, 3.29 -- Soup of the day, with a side of potato salad
Hotdog, 3.05 -- A hot dog, with saurkraut, relish, onions, topped with cheese
Steamed Veggies and Brown Rice, 3.99 -- Steamed vegetables over brown rice
Pasta, 3.89 -- Spaghetti with Marinara Sauce, and a slice of sourdough bread

%
```

First we iterate through the pancake menu.

And then the lunch menu, all with the same iteration code.

Chúng ta đã làm gì cho tới bây giờ?

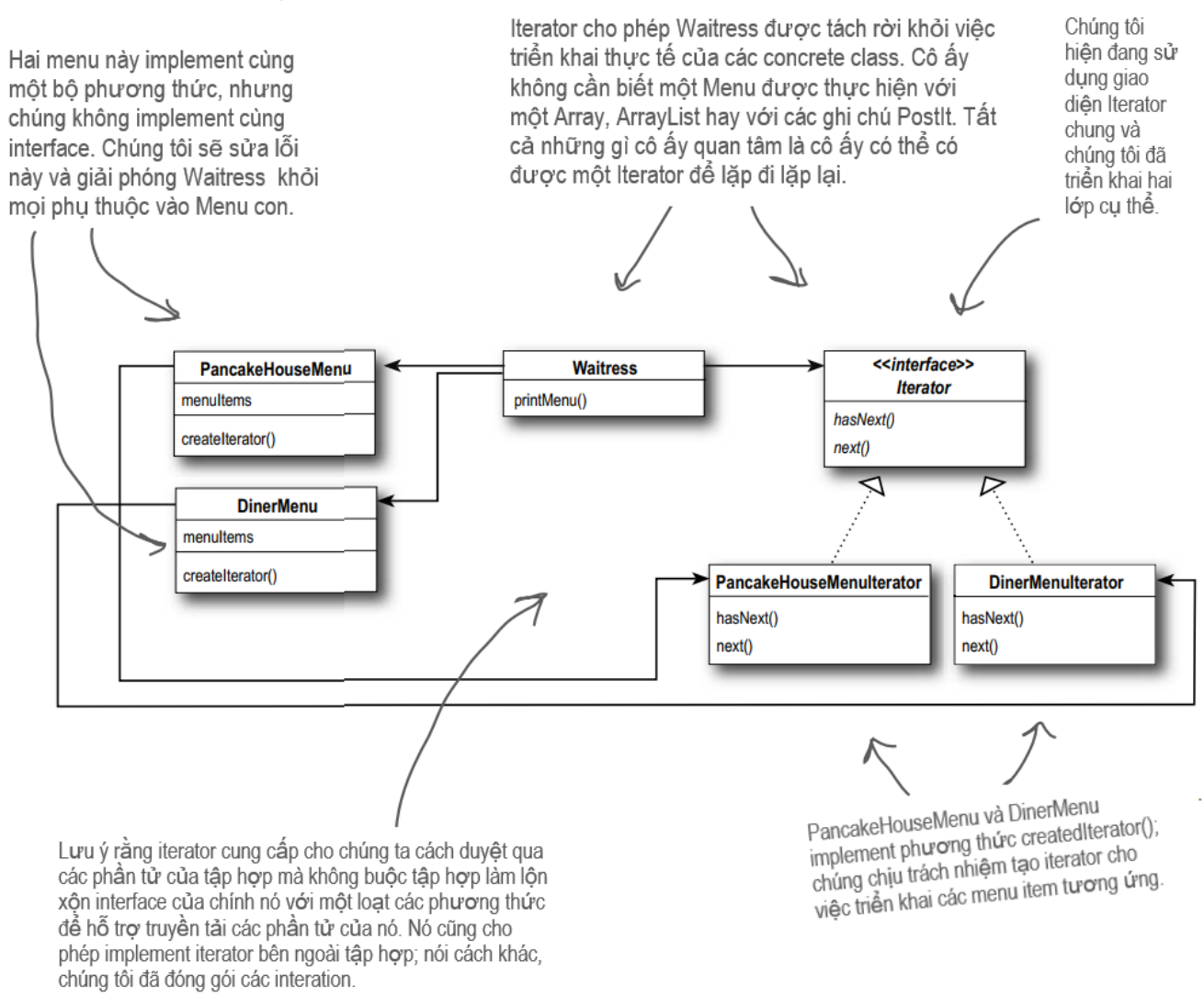
Để bắt đầu, chúng tôi đã làm cho các đầu bếp Objectville của chúng tôi rất hạnh phúc. Họ giải quyết sự khác biệt và giữ thực hiện riêng của họ. Khi chúng tôi đưa cho họ một **PancakeHouseMenuIterator** và **DinerMenuIterator**, tất cả những gì họ phải làm là thêm một phương thức **createIterator()** và họ đã hoàn thành.

Chúng tôi cũng đã tự giúp mình trong quá trình này. Waitress sẽ dễ dàng hơn nhiều để duy trì và mở rộng trong tương lai. Hãy đi qua chính xác những gì chúng ta đã làm và nghĩ về hậu quả:

Khó để bảo trì Waitress Implementation	Waitress với sức mạnh của Iterator
Các Menu không được đóng gói; chúng ta có thể thấy Diner đang sử dụng một Array và Pancake House sử dụng một ArrayList.	Việc triển khai Menu hiện được gói gọn. Waitress không biết các Menu giữ một tập hợp các menu item của chúng như thế nào.
Chúng ta cần hai vòng lặp để lặp qua MenuItems.	Tất cả những gì chúng ta cần là một vòng lặp đa hình xử lý bất kỳ bộ item nào miễn là nó implement Iterator
Waitress bị ràng buộc với các lớp cụ thể (MenuItem[] và ArrayList).	Waitress hiện sử dụng một interface (Iterator).
Waitress bị ràng buộc với hai lớp Menu cụ thể khác nhau, mặc dù giao diện của chúng gần như giống hệt nhau.	Các Menu interface bây giờ giống hệt nhau và, ồ, chúng ta vẫn chưa có interface chung, điều đó có nghĩa là Waitress vẫn bị ràng buộc với hai lớp Menu cụ thể. Chúng tôi khắc phục điều đó tốt hơn.

Những gì chúng ta có cho đến giờ…

Trước khi chúng tôi dọn dẹp mọi thứ, hãy có một cái nhìn toàn cảnh về thiết kế hiện tại của chúng tôi.

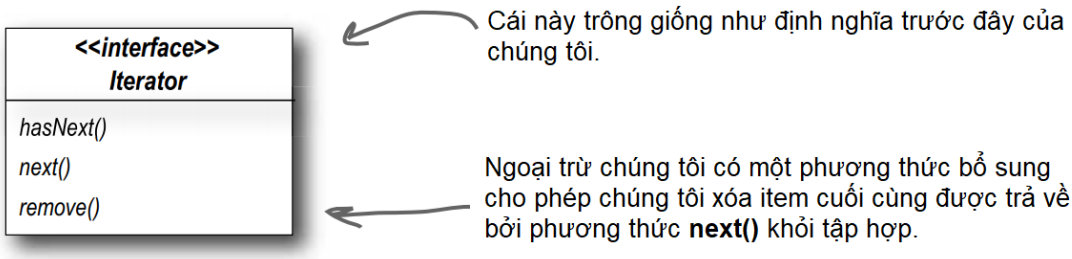


Cải tiến menu với Iterator Pattern...của Java

Được rồi, chúng tôi biết các interface của **PancakeHouseMenu** và **DinerMenu** hoàn toàn giống nhau và chúng tôi chưa định nghĩa interface chung cho chúng. Vì vậy, chúng tôi sẽ làm điều đó và thay đổi Waitress thêm một chút nữa.

Bạn có thể tự hỏi tại sao chúng tôi không sử dụng interface Iterator của Java từ đầu – chúng tôi đã làm vậy để bạn có thể thấy cách xây dựng một iterator. Bây giờ chúng tôi đã thực hiện điều đó, chúng tôi sẽ chuyển sang sử dụng interface Iterator Java, bởi vì chúng tôi sẽ nhận được rất nhiều lợi ích bằng cách implement nó thay vì interface Iterator "tự xây dựng" của chúng tôi. Những lợi ích gì? Bạn sẽ sớm thấy thôi.

Trước tiên, hãy xem qua giao diện **java.util.Iterator**:



Đây sẽ là một miếng bánh: Chúng ta chỉ cần thay đổi interface mà cả **PancakeHouseMenuIterator** và **DinerMenuIterator** extend, phải không? Hầu như … thực sự, nó thậm chí còn dễ dàng hơn thế. **Java.util** không chỉ có interface **Iterator** riêng mà ArrayList còn có phương thức **iterator()** trả về một iterator. Nói cách khác, chúng ta không bao giờ cần phải implement iterator riêng cho ArrayList. Tuy nhiên, chúng tôi vẫn cần triển khai cho **DinerMenu** vì giả sử nó phụ thuộc vào một mảng không hỗ trợ phương thức **iterator()** (hoặc không hỗ trợ bất kỳ cách nào khác để tạo một array iterator).

Không có câu hỏi ngớ ngẩn

Hỏi: Điều gì sẽ xảy ra nếu tôi không muốn cung cấp khả năng remove thứ gì đó khỏi tập hợp các đối tượng cơ bản?

Trả lời: Phương thức **remove()** được coi là tùy chọn. Bạn không phải cung cấp chức năng remove. Nhưng, rõ ràng là bạn cần phải cung cấp phương thức này vì nó là một phần của interface Iterator. Nếu bạn không cho phép **remove()** trong iterator của mình, bạn sẽ ném runtime exception **java.lang.UnsupportedOperationException**.

Tài liệu API của Iterator chỉ định rằng exception này có thể được ném từ **remove()** và bất kỳ thiết kế nào tốt đều sẽ kiểm tra exception này khi gọi phương thức **remove()**.

Hỏi: Làm thế nào để **remove()** hoạt động trong đa luồng (multiple thread) có thể sử dụng các iterator khác nhau trên cùng một tập hợp đối tượng?

Trả lời: Hành vi của **remove()** là không xác định nếu tập hợp bị thay đổi trong khi bạn đang duyệt qua nó. Vì vậy, bạn nên cẩn thận trong việc thiết kế code multiple thread của riêng bạn khi truy cập đồng thời một tập hợp.

Dọn dẹp mọi thứ với java.util.Iterator

Hãy bắt đầu với **PancakeHouseMenu**, thay đổi nó thành **java.util.Iterator** sẽ trở nên dễ dàng. Chúng tôi chỉ cần xóa lớp **PancakeHouseMenuIterator**, thêm **java.util.Iterator** lên đầu class **PancakeHouseMenu** và thay đổi một dòng của **PancakeHouseMenu**:

```
public Iterator createIterator() {
    return menuItems.iterator();
}
```

Thay vì tạo iterator của riêng chúng ta, bây giờ, chúng ta chỉ cần gọi phương thức **iterator()** trên menuItems ArrayList.

Và đó là nó, **PancakeHouseMenu** đã hoàn thành.

Bây giờ chúng ta cần thực hiện các thay đổi để cho phép **DinerMenu** hoạt động với **java.util.Iterator**.

```
import java.util.Iterator;

public class DinerMenuIterator implements Iterator {
    MenuItem[] list;
    int position = 0;

    public DinerMenuIterator(MenuItem[] list) {
        this.list = list;
    }

    public Object next() {
        //implementation here
    }

    public boolean hasNext() {
        //implementation here
    }

    public void remove() {
        if (position <= 0) {
            throw new IllegalStateException
                ("You can't remove an item until you've done at least one next()");
        }
        if (list[position-1] != null) {
            for (int i = position-1; i < (list.length-1); i++) {
                list[i] = list[i+1];
            }
            list[list.length-1] = null;
        }
    }
}
```

Đầu tiên chúng ta import java.util.Iterator, giao diện mà chúng tôi sẽ triển khai.

Không có thay đổi triển khai hiện tại của chúng tôi...

... nhưng chúng ta cần phải thực hiện **remove()**. Ở đây, vì đầu bếp đang sử dụng Mảng có kích thước cố định, chúng tôi chỉ cần chuyển tất cả các phần tử lên một đơn vị khi **remove()** được gọi.

Chúng ta gần hoàn thành...

Chúng ta chỉ cần cung cấp cho các Menu một interface chung và làm lại Waitress một chút. Giao diện Menu khá đơn giản: cuối cùng chúng tôi có thể muốn thêm một vài phương thức nữa, như **addItem()**, nhưng bây giờ chúng tôi sẽ cho phép các đầu bếp kiểm soát các menu của họ bằng cách đưa phương thức đó ra khỏi public interface:

```
public interface Menu {
    public Iterator createIterator();
}
```

Đây là một giao diện đơn giản chỉ cho phép client có được một iterator cho các phần tử trong menu.

Bây giờ chúng ta cần thêm implements Menu vào cả định nghĩa lớp **PancakeHouseMenu** và **DinerMenu**, sau đó cập nhật Waitress:




```
import java.util.Iterator;

public class Waitress {
    Menu pancakeHouseMenu;
    Menu dinerMenu;

    public Waitress(Menu pancakeHouseMenu, Menu dinerMenu) {
        this.pancakeHouseMenu = pancakeHouseMenu;
        this.dinerMenu = dinerMenu;
    }

    public void printMenu() {
        Iterator pancakeIterator = pancakeHouseMenu.createIterator();
        Iterator dinerIterator = dinerMenu.createIterator();
        System.out.println("MENU\n----\nBREAKFAST");
        printMenu(pancakeIterator);
        System.out.println("\nLUNCH");
        printMenu(dinerIterator);
    }

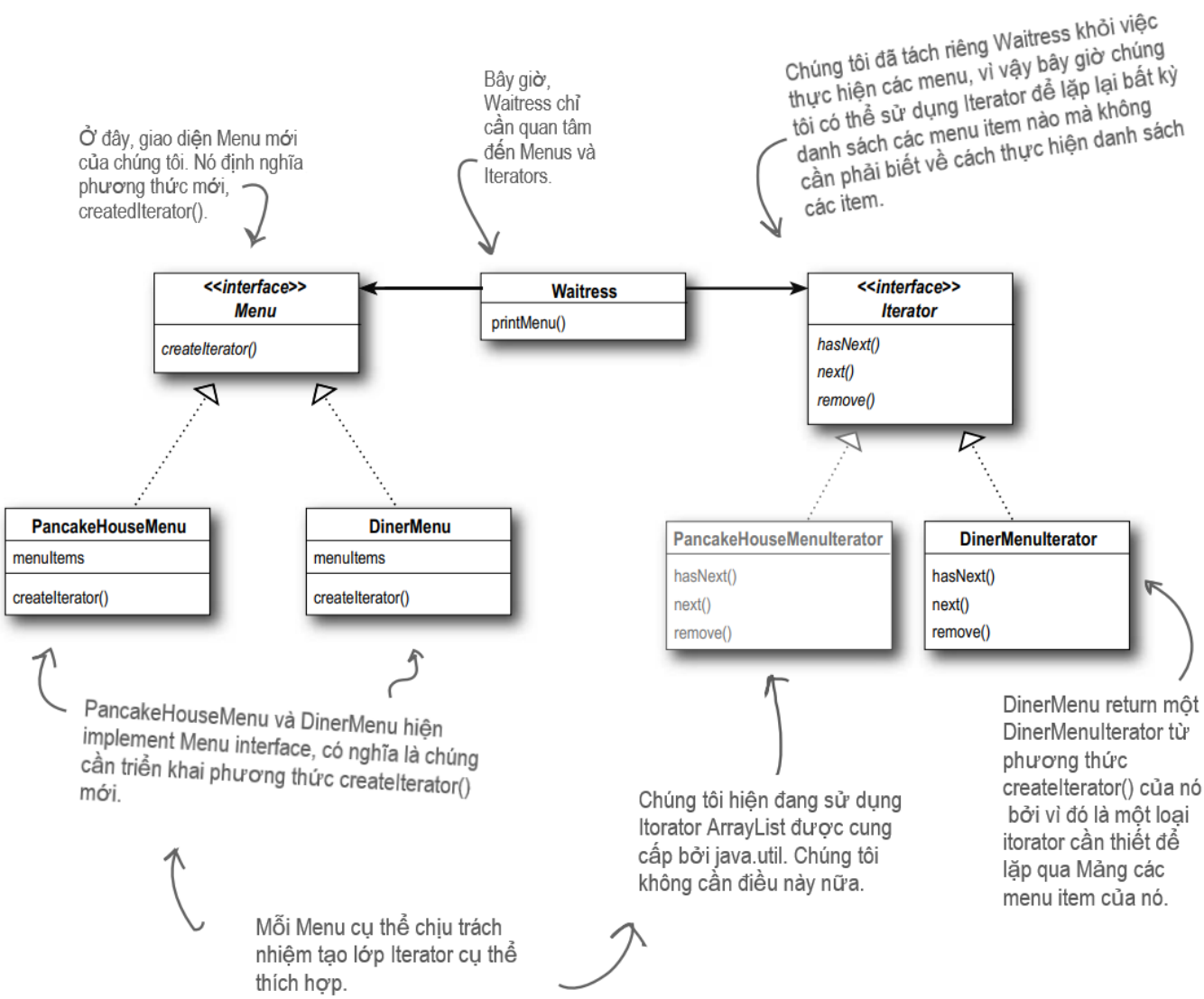
    private void printMenu(Iterator iterator) {
        while (iterator.hasNext()) {
            MenuItem menuItem = (MenuItem)iterator.next();
            System.out.print(menuItem.getName() + ", ");
            System.out.print(menuItem.getPrice() + " -- ");
            System.out.println(menuItem.getDescription());
        }
    }

    // other methods here
}
```

Iterator Pattern cho chúng ta những gì?

Các lớp **PancakeHouseMenu** và **DinerMenu** implement cùng một interface, Menu. Waitress có thể tham chiếu từng đối tượng menu bằng interface chứ không phải lớp cụ thể. Vì vậy, chúng tôi giảm bớt sự phụ thuộc giữa Waitress và các lớp cụ thể bằng cách lập trình vào một giao diện, chứ không phải là một triển khai (programming to an interface, not an implementation) => Điều này giải quyết vấn đề của Waitress phụ thuộc vào concrete Menu (menu con).

Interface Menu mới có một phương thức, **createIterator()**, được triển khai bởi **PancakeHouseMenu** và **DinerMenu**. Mỗi lớp menu đảm nhận trách nhiệm tạo ra một Iterator cụ thể phù hợp cho việc thực hiện bên trong các menu item => Điều này giải quyết vấn đề của Waitress phụ thuộc vào việc triển khai MenuItem.



Định nghĩa Iterator Pattern

Bạn đã thấy cách triển khai **Iterator Pattern** với iterator riêng của mình. Bạn cũng đã thấy cách Java hỗ trợ các iterator trong một số lớp collection (ArrayList). Bây giờ đã đến lúc để kiểm tra định nghĩa chính thức của mẫu:

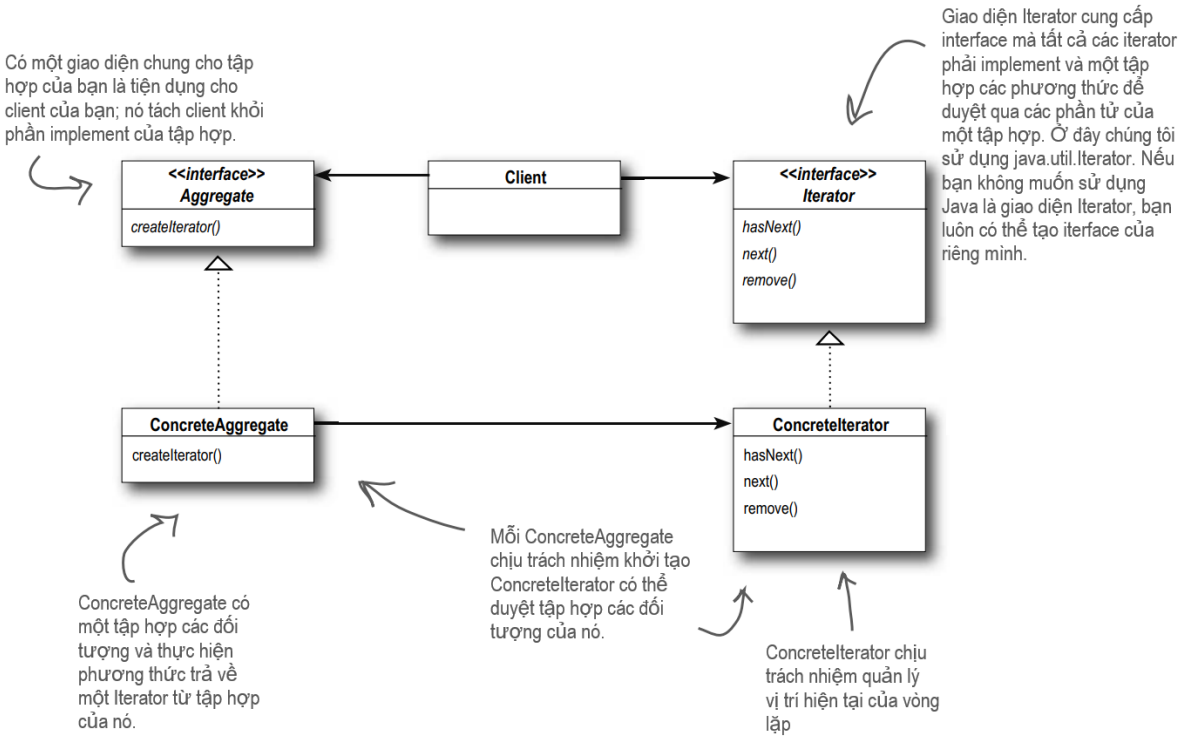
> **The Iterator Pattern** provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

(Iterator Pattern cung cấp một cách để truy cập các phần tử của một tập hợp đối tượng một cách tuần tự mà không làm lộ đại diện bên dưới của nó)

Điều này rất có ý nghĩa: mẫu cung cấp cho bạn một cách để duyệt qua các phần tử của tập hợp mà không cần phải biết cách mọi thứ được thể hiện bên trong. Bạn đã thấy điều đó với hai cài đặt của Menu. Nhưng hiệu quả của việc sử dụng các **Iterator Pattern** trong thiết kế của bạn là: một khi bạn có cách truy cập thống nhất các phần tử của tất cả các tập hợp của mình, bạn có thể viết code đa hình hoạt động với bất kỳ tập hợp nào – giống như phương thức **printMenu()**, nó không quan tâm các menu item được lưu trong một Array hay ArrayList (hoặc bất cứ thứ gì khác có thể tạo Iterator), miễn là nó có thể tạo được Iterator.

Tác động quan trọng khác đối với thiết kế của bạn là **Iterator Pattern** có trách nhiệm di chuyển các phần tử và giao trách nhiệm đó cho iterator object chứ không phải collection object. Điều này không chỉ giữ cho collection interface và collection implement (các đối tượng Array, ArrayList) đơn giản hơn, nó loại bỏ trách nhiệm duyệt phần tử khỏi tập hợp và giữ cho tập hợp tập trung vào những thứ cần tập trung (quản lý tập hợp đối tượng), không phải quản lý vòng lặp.

Iterator Pattern: Sơ đồ lớp



Sử dụng sức mạnh bộ não

Sơ đồ lớp cho **Iterator Pattern** trông rất giống với Mẫu nào khác mà bạn đã nghiên cứu? Gợi ý: Một lớp con quyết định đối tượng nào được tạo.

Không có câu hỏi ngớ ngẩn

Hỏi: Tôi đã thấy các cuốn sách khác biểu diễn sơ đồ lớp Iterator với các phương thức **first()**, **next()**, **isDone()** và **currentItem()**. Tại sao chúng không giống các phương thức của **Iterator Pattern** trong chương này?

Trả lời: Đó là những tên phương thức cũ đã được sử dụng. Các tên này đã thay đổi theo thời gian và bây giờ chúng ta có **next()**, **hasNext()** và thậm chí **remove()** trong **java.util.Iterator**.

Hãy nhìn vào các phương thức cũ **next()** và **currentItem()** đã được hợp nhất thành một phương thức trong **java.util**. Phương thức **isDone()** rõ ràng đã trở thành **hasNext()**; nhưng chúng ta không có phương thức tương ứng với **First()**. Điều đó bởi vì trong Java, chúng ta có xu hướng chỉ cần có một vòng lặp mới bất cứ khi nào chúng ta cần bắt đầu duyệt phần tử. Tuy nhiên, bạn có thể thấy có rất ít sự khác biệt trong các giao diện này. Trong thực tế, có một loạt các hành vi bạn có thể đưa ra cho các iterator của mình. Phương thức **remove()** là một ví dụ về phần mở rộng trong **java.util.Iterator**.

Hỏi: Tôi đã nghe nói về các "internal" iterator và "external" iterator. Chúng là gì? Chúng ta đã thực hiện trong ví dụ trên loại nào?

Trả lời: Chúng ta đã triển khai một external iterator, có nghĩa là client điều khiển phép lặp bằng cách gọi **next()** để lấy phần tử tiếp theo. Một internal iterator được điều khiển bởi chính iterator đó. Trong trường hợp đó, bởi vì nó là bộ lặp iterator mà duyệt qua các phần tử, bạn phải nói cho iterator biết phải làm gì với các phần tử đó khi nó duyệt qua chúng. Điều đó có nghĩa là bạn cần một cách để chuyển một hoạt động đến một iterator. Các internal iterator kém linh hoạt hơn các external iterator vì client không có quyền kiểm soát vòng lặp. Tuy nhiên, một số người có thể lập luận rằng chúng dễ sử dụng hơn vì bạn chỉ cần đưa cho chúng một thao tác và bảo chúng lặp lại, và chúng làm tất cả công việc cho bạn.

Hỏi: Tôi có thể triển khai một Iterator có thể duyệt lùi giống như duyệt tới không?

Trả lời: Chắc chắn rồi. Trong trường hợp đó, bạn có thể muốn thêm hai phương thức, một phương thức để duyệt đến phần tử phía sau và một phương thức để duyệt đến phần tử phía trước. Java's Collection Framework cung cấp một loại giao diện lặp khác gọi là **ListIterator**. Trình lặp này thêm **previous()** và một vài phương thức khác vào giao diện Iterator tiêu chuẩn. Nó được hỗ trợ bởi bất kỳ tập hợp nào implement List interface.

Hỏi: Ai định nghĩa thứ tự duyệt phần tử trong tập hợp như **Hashtable**, chúng vốn không có thứ tự?

Trả lời: Iterator ngầm định là không có thứ tự. Các tập hợp cơ bản có thể không được sắp xếp; chúng thậm chí có thể chứa các phần tử trùng lặp. Vì vậy, thứ tự có liên quan đến cả các thuộc tính của tập hợp bên trong các các implement của nó. Nói chung, bạn không nên đưa ra giả định nào về thứ tự trừ khi Collection chỉ ra.

Hỏi: Bạn nói rằng chúng ta có thể viết code đa hình (polymorphic code) bằng cách sử dụng một trình iterator; bạn có thể giải thích thêm không?

Trả lời: Khi chúng ta viết các phương thức lấy Iterator làm tham số, chúng ta đang sử dụng phép lặp đa hình. Điều đó có nghĩa là chúng tôi đang tạo code có thể duyệt trên bất kỳ tập hợp nào miễn là nó hỗ trợ Iterator. Chúng tôi không quan tâm đến cách tập hợp được implement, chúng tôi vẫn có thể viết code để duyệt nó.

Hỏi: Nếu tôi sử dụng Java, không phải lúc nào tôi cũng muốn sử dụng giao diện **java.util.Iterator**, vì vậy tôi có thể tạo ra các iterator của riêng mình cho các lớp đã sử dụng Java iterator không?

Trả lời: Có lẽ. Nếu bạn có một Iterator interface chung, chắc chắn nó sẽ giúp bạn dễ dàng trộn và kết nối các tập hợp của riêng bạn với các tập hợp Java như ArrayList và Vector. Nhưng hãy nhớ, nếu bạn cần thêm chức năng vào giao diện Iterator cho tập hợp của bạn, bạn luôn có thể extend Iterator interface.

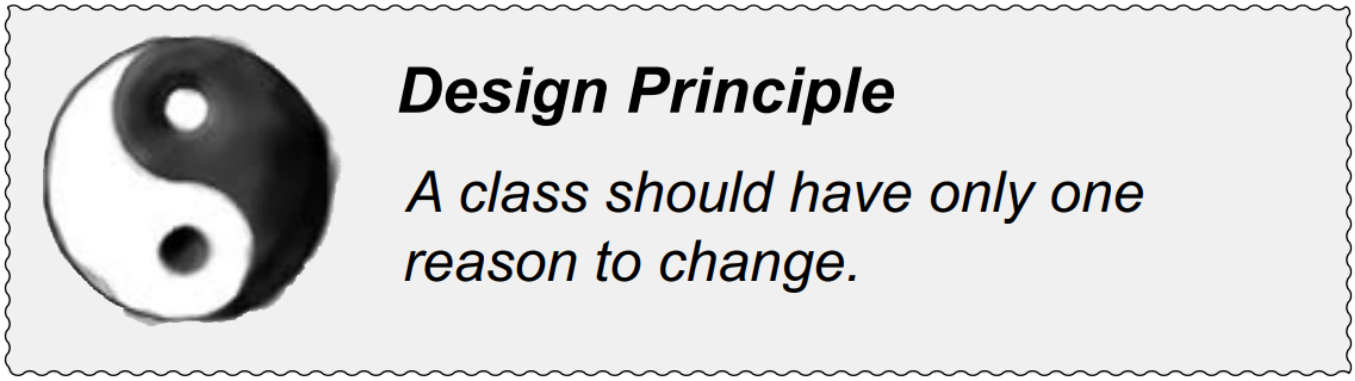
Hỏi: Tôi đã thấy một Enumeration interface trong Java; Nó có cài đặt **Iterator Pattern** không?

Trả lời: Chúng tôi đã nói về điều này trong Chương Adaptor Pattern (https://toihocdesignpattern.com/chuong-7-adapter-va-facade-pattern-tro-nen-thich-nghi-phan-1.html). Nhớ lại xem? Các **java.util.Enumeration** là một triển khai cũ hơn của Iterator đã được thay thế bởi **java.util.Iterator**. Enumeration có hai phương thức, **hasMoreElements()**, tương ứng với **hasNex()** và **nextElement()** tương ứng với **next()**. Tuy nhiên, bạn có thể muốn sử dụng Iterator trên Enumeration vì nhiều lớp Java hỗ trợ nó. Nếu bạn cần chuyển đổi từ cái này sang cái khác, hãy xem lại Chương Adaptor (https://toihocdesignpattern.com/chuong-7-adapter-va-facade-pattern-tro-nen-thich-nghi-phan-1.html) nơi bạn đã triển khai bộ chuyển đổi Enumeration và Iterator.

Single Responsibility và Iterator Pattern

Điều gì xảy ra nếu chúng tôi cho phép tập hợp của chúng tôi implement các “internal collection” và các chức năng liên quan VÀ các iteration method của chúng? Chà, chúng ta đã biết rằng sẽ mở rộng số lượng phương thức trong tập hợp, nhưng vậy thì sao? Tại sao điều đó sẽ rất tệ?

Chà, để xem tại sao, trước tiên bạn cần nhận ra rằng khi chúng tôi cho phép một lớp không chỉ quan tâm nghiệp vụ của riêng mình (quản lý một số loại tập hợp) mà còn đảm nhận nhiều trách nhiệm hơn (như vòng lặp) thì chúng tôi đã giao cho lớp hai lý do để thay đổi. Hai ư? vâng, hai: nó có thể thay đổi nếu **bộ sưu tập thay đổi** theo một cách nào đó và nó có thể thay đổi nếu **cách chúng ta duyệt vòng lặp thay đổi**. Vì vậy, một lần nữa người bạn THAY ĐỔI của chúng tôi là trung tâm của một nguyên tắc thiết kế khác:



Mỗi lớp nên có một và chỉ một lý do để thay đổi.

Mỗi trách nhiệm của một lớp là một lĩnh vực thay đổi tiềm năng. Nhiều hơn một trách nhiệm có nghĩa là nhiều hơn một lĩnh vực thay đổi.

Nguyên tắc này hướng dẫn chúng tôi giữ cho mỗi lớp chịu trách nhiệm duy nhất.

Chúng tôi biết rằng chúng tôi muốn tránh thay đổi trong một lớp như "bệnh dịch" – code sửa đổi cung cấp tất cả các loại cơ hội cho các vấn đề xuất hiện. Có hai cách để có thể thay đổi lớp, sẽ tăng xác suất lớp đó bị thay đổi trong tương lai và khi đó, nó sẽ xảy ra và ảnh hưởng đến hai khía cạnh của thiết kế của bạn.

Giải pháp? Nguyên tắc hướng dẫn chúng tôi phân công mỗi trách nhiệm cho một lớp và chỉ một lớp.

Điều đó đúng, nó có thể dễ dàng như vậy, và một lần nữa, nó không phải là: tách biệt trách nhiệm trong thiết kế là một trong những điều khó khăn nhất để làm. Bộ não của chúng ta chỉ quá giỏi trong việc nhìn thấy một tập hợp các hành vi và nhóm chúng lại với nhau ngay cả khi thực sự có hai hoặc nhiều trách nhiệm. Cách duy nhất để thành công là siêng năng kiểm tra các thiết kế của bạn và tìm ra các dấu hiệu cho thấy một lớp bị thay đổi theo nhiều cách khi hệ thống của bạn phát triển.

Sự gắn kết (Cohesion) là một thuật ngữ mà bạn nghe thấy được sử dụng như một thước đo mức độ chặt chẽ của một lớp hoặc một mô-đun hỗ trợ cho một mục đích hoặc trách nhiệm duy nhất.

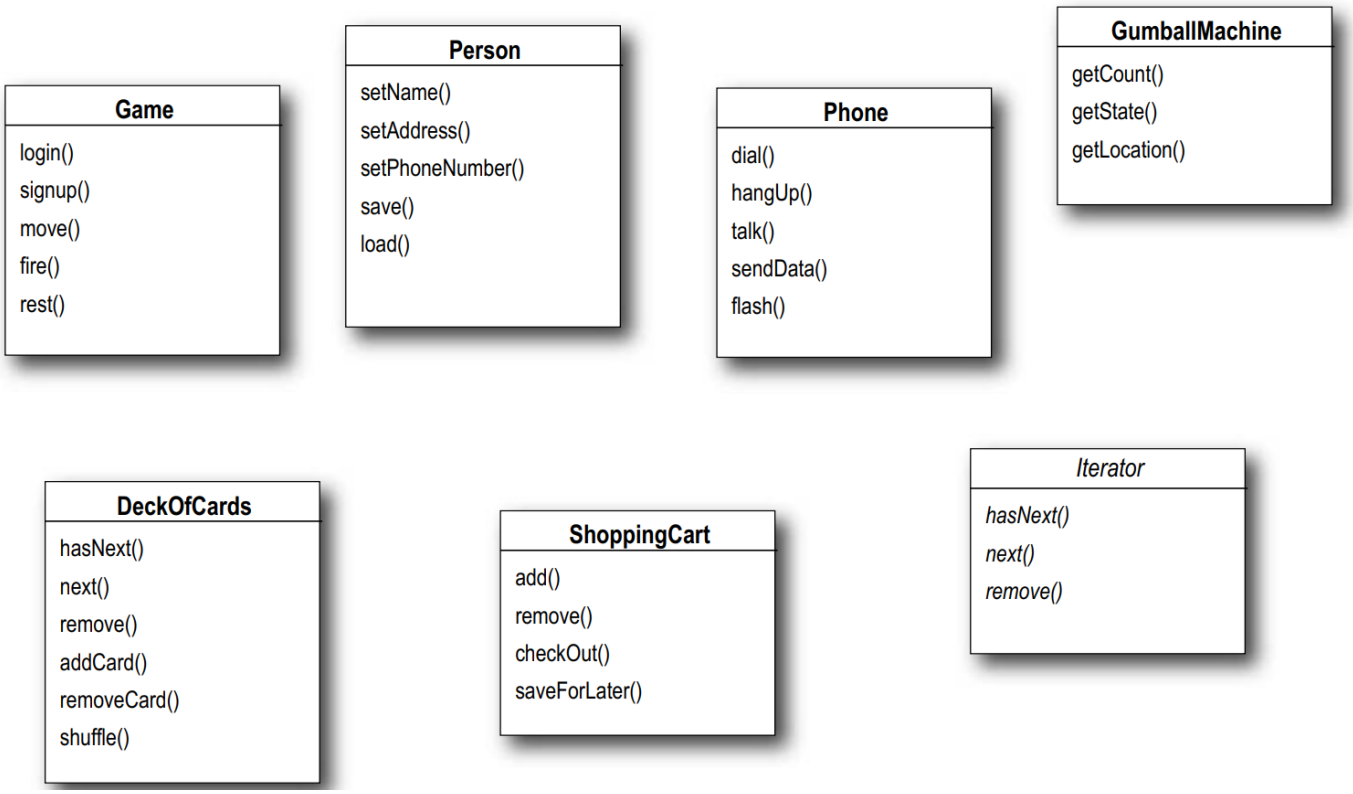
Chúng tôi nói rằng một mô-đun hoặc lớp có độ gắn kết cao khi nó được thiết kế xung quanh một tập hợp các hàm liên quan và chúng tôi nói rằng nó có độ gắn kết thấp khi nó được thiết kế xung quanh một tập hợp các hàm không liên quan.

Sự gắn kết là một khái niệm tổng quát hơn **Nguyên tắc đơn trách nhiệm**, nhưng hai nguyên tắc này có liên quan chặt chẽ với nhau.

Các lớp tuân thủ **Nguyên tắc đơn trách nhiệm** có xu hướng có sự gắn kết cao và dễ duy trì hơn các lớp đảm nhận nhiều trách nhiệm và có độ gắn kết thấp.

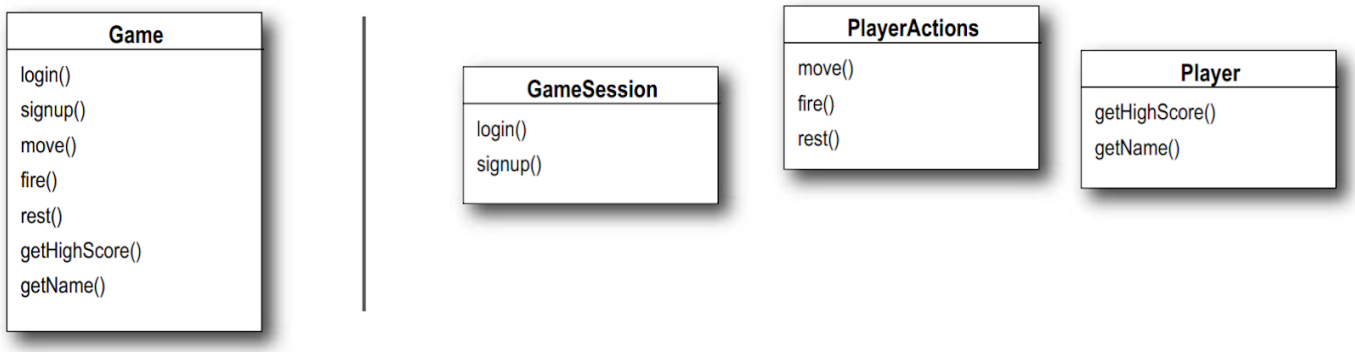
Sử dụng sức mạnh bộ não

Kiểm tra các lớp này và xác định cái nào có nhiều trách nhiệm.



Sử dụng sức mạnh bộ não 2

Xác định xem các lớp này có độ gắn kết thấp hay cao.

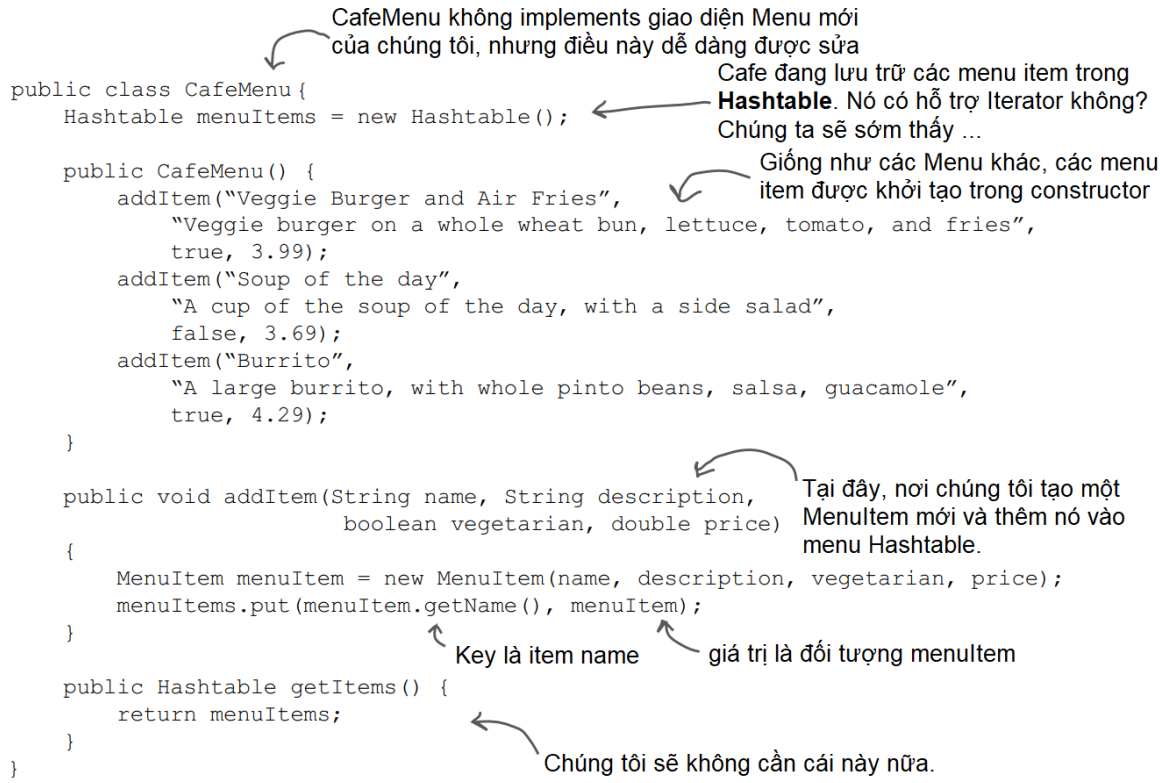


Thật tốt khi bạn đang tìm hiểu về **Iterator Pattern** bởi vì tôi vừa nghe rằng Objectville Mergers và Acquisitions đã thực hiện một thỏa thuận khác … đưa Objectville Cafe vào Dinner menu của họ.



Hãy nhìn vào CafeMenu

Ở đây là **CafeMenu**. Có vẻ như rất nhiều rắc rối khi tích hợp **CafeMenu** vào khuôn khổ của chúng tôi… Hãy kiểm tra nó:




```
public class CafeMenu {
    Hashtable menuItems = new Hashtable();

    public CafeMenu() {
        addItem("Veggie Burger and Air Fries",
            "Veggie burger on a whole wheat bun, lettuce, tomato, and fries",
            true, 3.99);
        addItem("Soup of the day",
            "A cup of the soup of the day, with a side salad",
            false, 3.69);
        addItem("Burrito",
            "A large burrito, with whole pinto beans, salsa, guacamole",
            true, 4.29);
    }

    public void addItem(String name, String description,
                        boolean vegetarian, double price)
    {
        MenuItem menuItem = new MenuItem(name, description, vegetarian, price);
        menuItems.put(menuItem.getName(), menuItem);
    }

    public Hashtable getItems() {
        return menuItems;
    }
}
```

Bài tập

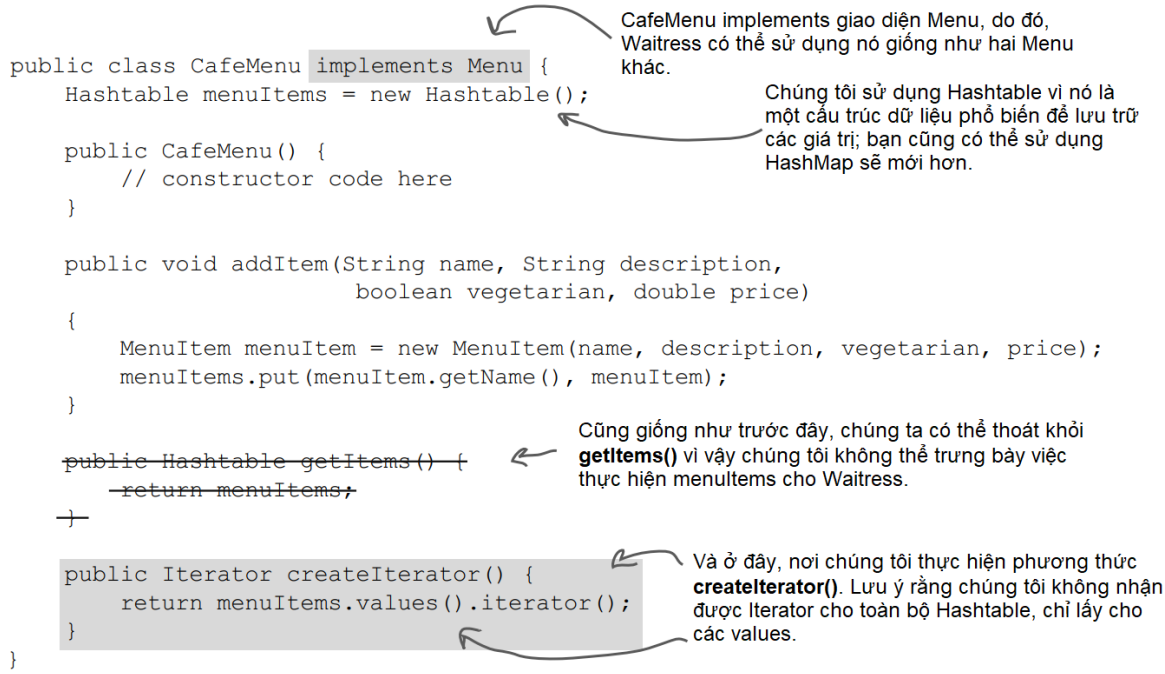
Trước khi nhìn vào trang tiếp theo, hãy nhanh chóng ghi lại ba điều chúng ta phải làm với đoạn code này để phù hợp với framework của chúng ta.

Đáp án:

1. Implement Menu interface.
2. Thoát khỏi getItems().
3. Thêm createIterator() và return một Iterator có thể duyệt qua các giá trị Hashtable.

Viết lại code CafeMenu

Đưa **CafeMenu** vào khuôn khổ của chúng tôi rất dễ dàng. Tại sao? Bởi vì **Hashtable** là một trong những bộ sưu tập Java hỗ trợ **Iterator**. Nhưng nó không hoàn toàn giống với ArrayList…




```
public class CafeMenu implements Menu {
    Hashtable menuItems = new Hashtable();

    public CafeMenu() {
        // constructor code here
    }

    public void addItem(String name, String description,
                        boolean vegetarian, double price)
    {
        MenuItem menuItem = new MenuItem(name, description, vegetarian, price);
        menuItems.put(menuItem.getName(), menuItem);
    }

    public Hashtable getItems() {
        return menuItems;
    }

    public Iterator createIterator() {
        return menuItems.values().iterator();
    }
}
```

Hashtable phức tạp hơn một chút so với ArrayList vì nó hỗ trợ cả key và value, nhưng chúng ta vẫn có thể nhận được Iterator cho các value (chúng là các **MenuItem**).

```
public Iterator createIterator() {
    return menuItems.values().iterator();
}
```

Đầu tiên chúng ta nhận được các giá trị của Hashtable, đây chỉ là một tập hợp tất cả các đối tượng trong hashtable.

May mắn thay, bộ sưu tập đó hỗ trợ phương thức iterator(), trả về một đối tượng kiểu **java.util.Iterator**.

Thêm CafeMenu vào Waitress

Điều đó thật dễ dàng; Làm thế nào về việc sửa đổi Waitress để hỗ trợ Menu mới của chúng tôi? Bây giờ, Waitress mong đợi Iterator, điều đó cũng dễ dàng.

```
public class Waitress {
    Menu pancakeHouseMenu;
    Menu dinerMenu;
    Menu cafeMenu;

    public Waitress(Menu pancakeHouseMenu, Menu dinerMenu, Menu cafeMenu) {
        this.pancakeHouseMenu = pancakeHouseMenu;
        this.dinerMenu = dinerMenu;
        this.cafeMenu = cafeMenu;
    }

    public void printMenu() {
        Iterator pancakeIterator = pancakeHouseMenu.createIterator();
        Iterator dinerIterator = dinerMenu.createIterator();
        Iterator cafeIterator = cafeMenu.createIterator();
        System.out.println("MENU\n----\nBREAKFAST");
        printMenu(pancakeIterator);
        System.out.println("\nLUNCH");
        printMenu(dinerIterator);
        System.out.println("\nDINNER");
        printMenu(cafeIterator);
    }

    private void printMenu(Iterator iterator) {
        while (iterator.hasNext()) {
            MenuItem menuItem = (MenuItem)iterator.next();
            System.out.print(menuItem.getName() + ", ");
            System.out.print(menuItem.getPrice() + " -- ");
            System.out.println(menuItem.getDescription());
        }
    }
}
```

Menu Café được chuyển vào Waitress trong hàm khởi tạo với các menu khác và chúng ta bỏ nó vào một biến instance.

Chúng tôi đã sử dụng menu Café cho thực đơn bữa tối của mình. Tất cả những gì chúng ta phải làm để in nó là tạo iterator và chuyển nó tới **printMenu()**.

Không có thay đổi ở đây

Bữa sáng, bữa trưa VÀ bữa tối

Hãy để cập nhật test drive của chúng ta để đảm bảo tất cả đều hoạt động đúng.

```
public class MenuTestDrive {
    public static void main(String args[]) {
        PancakeHouseMenu pancakeHouseMenu = new PancakeHouseMenu();
        DinerMenu dinerMenu = new DinerMenu();
        CafeMenu cafeMenu = new CafeMenu();

        Waitress waitress = new Waitress(pancakeHouseMenu, dinerMenu, cafeMenu);

        waitress.printMenu();
}
```

Create a CafeMenu...

... and pass it to the waitress.

Now, when we print we should see all three menus.

Tại đây, chạy thử nghiệm; kiểm tra thực đơn bữa tối mới từ Cafe!

```
File  Edit  Window  Help  Kathy&BertLikePancakes
% java DinerMenuTestDrive

MENU
----
BREAKFAST
K&B's Pancake Breakfast, 2.99 -- Pancakes with scrambled eggs, and toast
Regular Pancake Breakfast, 2.99 -- Pancakes with fried eggs, sausage
Blueberry Pancakes, 3.49 -- Pancakes made with fresh blueberries
Waffles, 3.59 -- Waffles, with your choice of blueberries or strawberries

LUNCH
Vegetarian BLT, 2.99 -- (Fakin') Bacon with lettuce & tomato on whole wheat
BLT, 2.99 -- Bacon with lettuce & tomato on whole wheat
Soup of the day, 3.29 -- Soup of the day, with a side of potato salad
Hotdog, 3.05 -- A hot dog, with saurkraut, relish, onions, topped with cheese
Steamed Veggies and Brown Rice, 3.99 -- Steamed vegetables over brown rice
Pasta, 3.89 -- Spaghetti with Marinara Sauce, and a slice of sourdough bread

DINNER
Soup of the day, 3.69 -- A cup of the soup of the day, with a side salad
Burrito, 4.29 -- A large burrito, with whole pinto beans, salsa, guacamole
Veggie Burger and Air Fries, 3.99 -- Veggie burger on a whole wheat bun,
 lettuce, tomato, and fries
%
```

First we iterate through the pancake menu.

And then the diner menu.

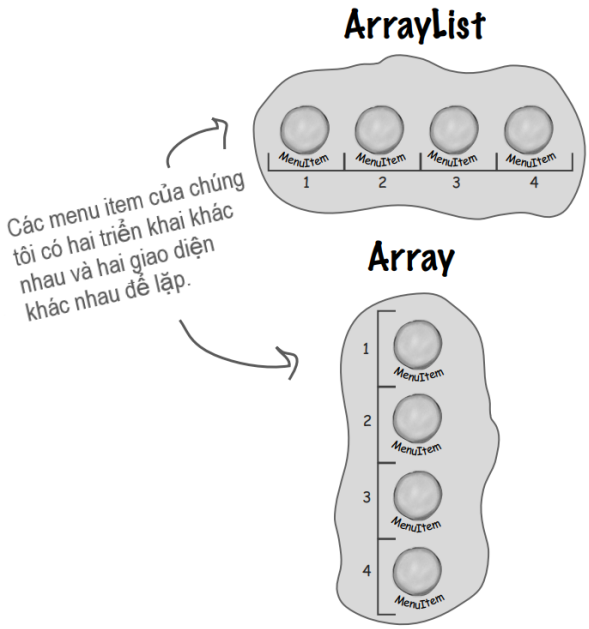
And finally the new café menu, all with the same iteration code.

Chúng ta đã làm gì?



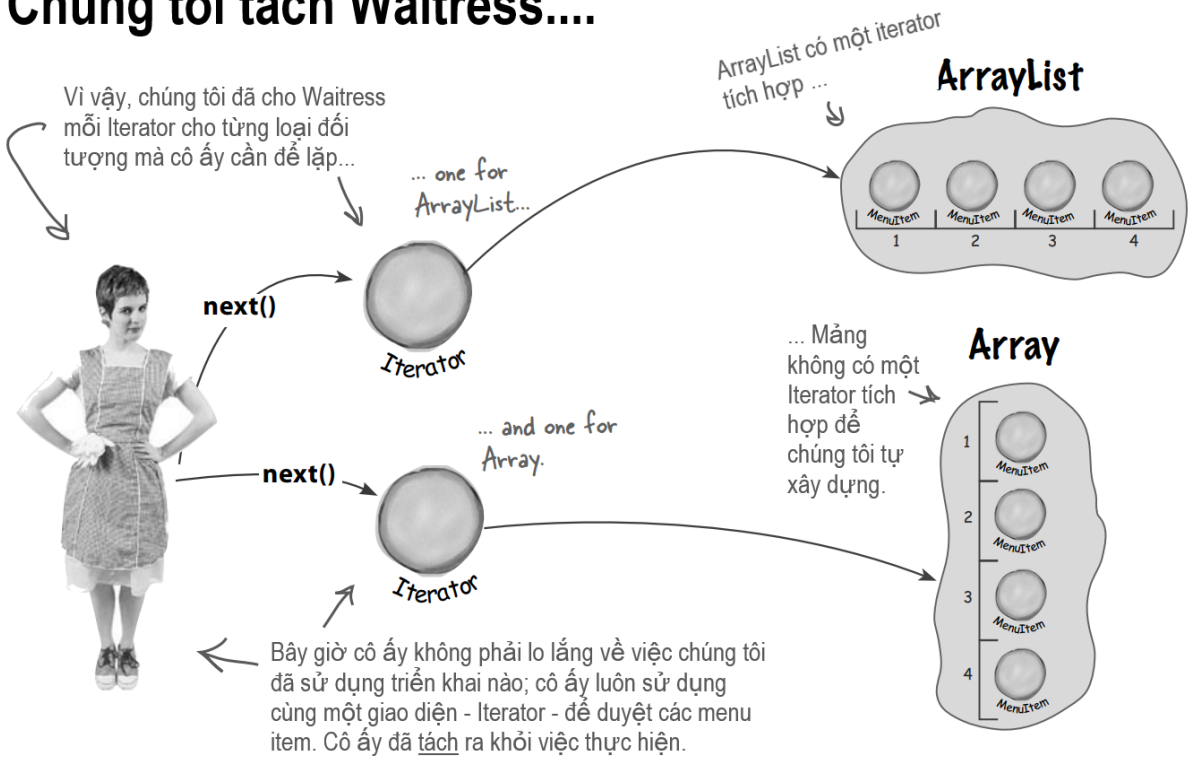
Chúng tôi muốn cung cấp cho Người phục vụ một cách dễ dàng để duyệt qua các item trong menu ...

... và chúng tôi đã không muốn cô ấy biết về cách thực hiện các menu item.

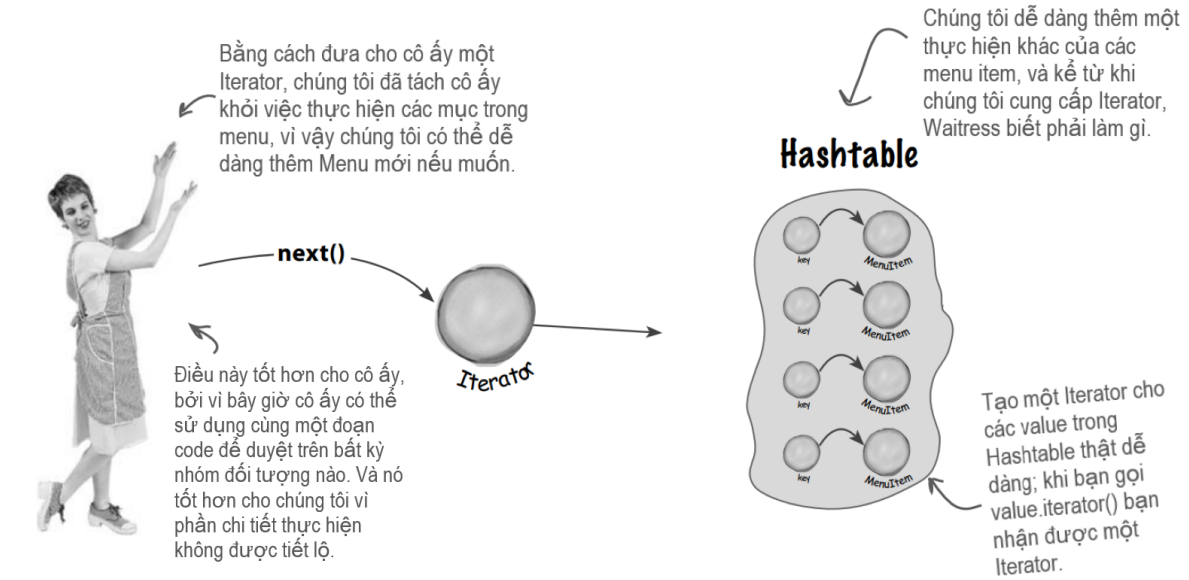


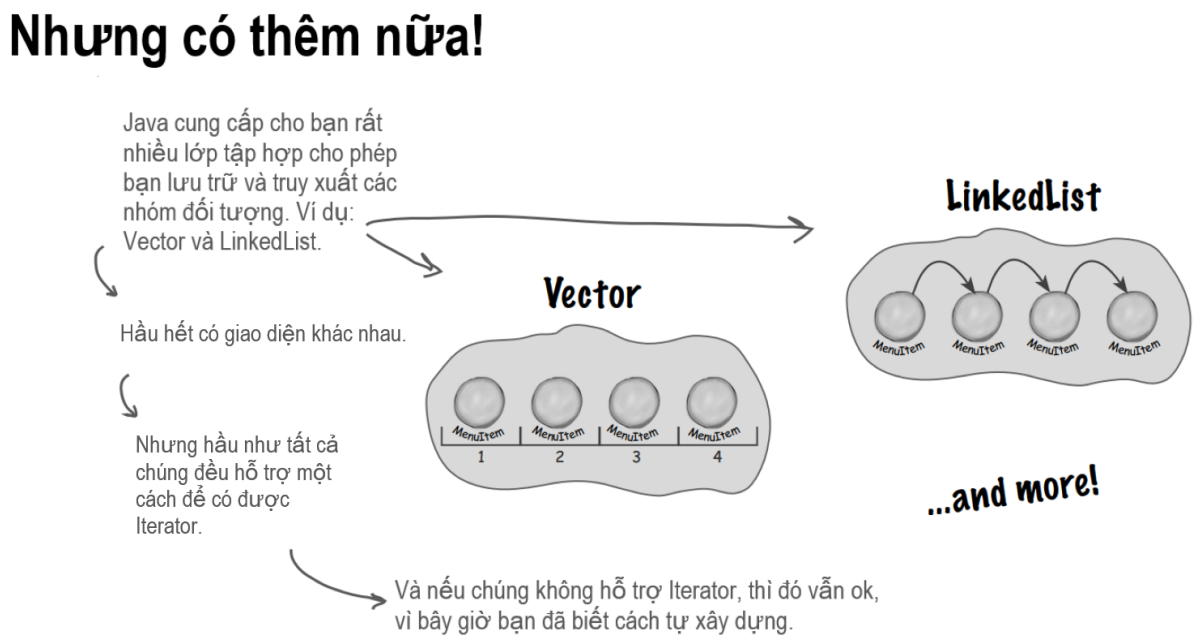
Các menu item của chúng tôi có hai triển khai khác nhau và hai giao diện khác nhau để lặp.

Chúng tôi tách Waitress....



...và chúng tôi đã làm cho Waitress mở rộng hơn



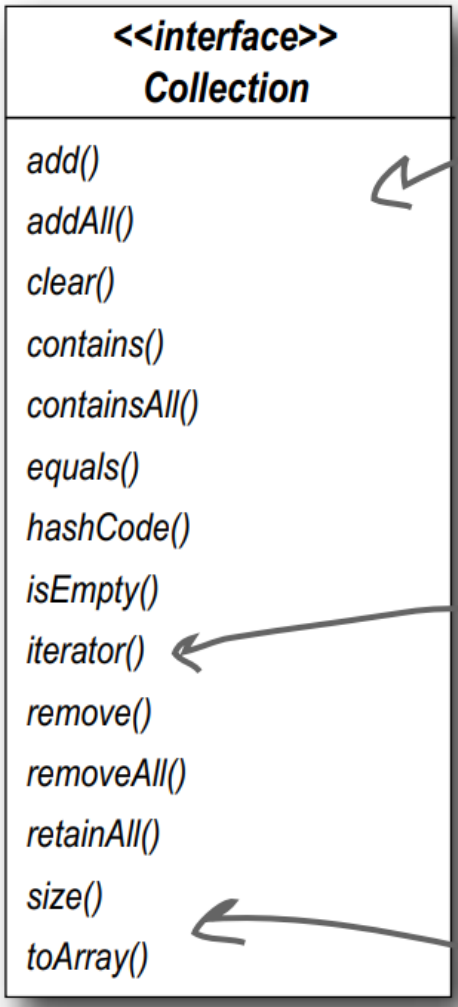


Iterator Pattern và Collection

Chúng tôi đã sử dụng một vài lớp là một phần của Java Collections Framework. Framework này chỉ là một tập hợp các lớp và interface, bao gồm ArrayList mà chúng tôi đã sử dụng và nhiều thứ khác như Vector, LinkedList, Stack và PriorityQueue. Các lớp này implement giao diện **java.util.Collection**, chứa một loạt các phương thức hữu ích để thao tác các nhóm đối tượng.

Hashtable là một trong một vài lớp gián tiếp hỗ trợ Iterator. Như bạn đã thấy khi chúng tôi triển khai **CafeMenu**, bạn có thể nhận được Iterator từ nó, nhưng chỉ bằng cách truy xuất các giá trị được gọi là Collection của nó. Nếu bạn nghĩ về nó, điều này có ý nghĩa: Hashtable chứa hai bộ đối tượng: keys và values. Nếu chúng ta muốn lặp các value của nó, trước tiên chúng ta cần truy xuất chúng từ Hashtable, sau đó lấy iterator.

Hãy xem nhanh giao diện:



Như bạn có thể thấy, có tất cả các loại công cụ tốt ở đây. Bạn có thể thêm và xóa các phần tử khỏi tập hợp của mình mà không cần biết cách thức triển khai.

Ở đây, người bạn cũ của chúng ta, phương thức iterator(). Với phương thức này, bạn có thể nhận được một Iterator cho bất kỳ lớp nào implement giao diện Collection.

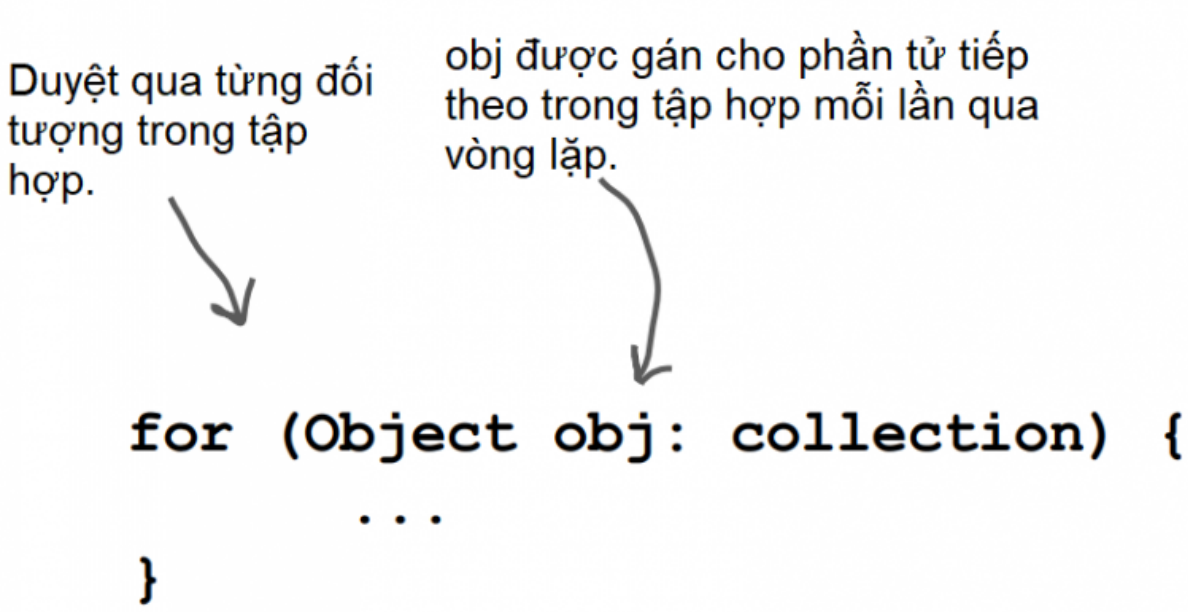
Các phương thức tiện dụng khác bao gồm **size()**, để lấy số lượng phần tử và **toArray()** để biến tập hợp của bạn thành một mảng.

Điều thú vị về Collections và Iterator là mỗi đối tượng Collection đều biết cách tạo Iterator của riêng mình. Gọi iterator() trên ArrayList trả về một Iterator cụ thể được tạo cho ArrayLists, nhưng bạn không bao giờ cần phải xem hoặc lo lắng về lớp cụ thể mà nó sử dụng; bạn chỉ cần sử dụng interface Iterator.



Java 5 bao gồm một dạng mới của câu lệnh **for**, được gọi là **for/in**, cho phép bạn lặp lại một tập hợp hoặc một mảng mà không cần tạo một vòng lặp rõ ràng.

Để sử dụng **for/in**, bạn sử dụng câu lệnh for trông giống như:



Đây là cách bạn duyệt qua một ArrayList bằng cách sử dụng **for/in**:

```
ArrayList items = new ArrayList();
items.add(new MenuItem("Pancakes", "delicious pancakes", true, 1.59);
items.add(new MenuItem("Waffles", "yummy waffles", true, 1.99);
items.add(new MenuItem("Toast", "perfect toast", true, 0.59);

for (MenuItem item: items) {
    System.out.println("Breakfast item: " + item);
}
```

Load up an ArrayList of MenuItems.

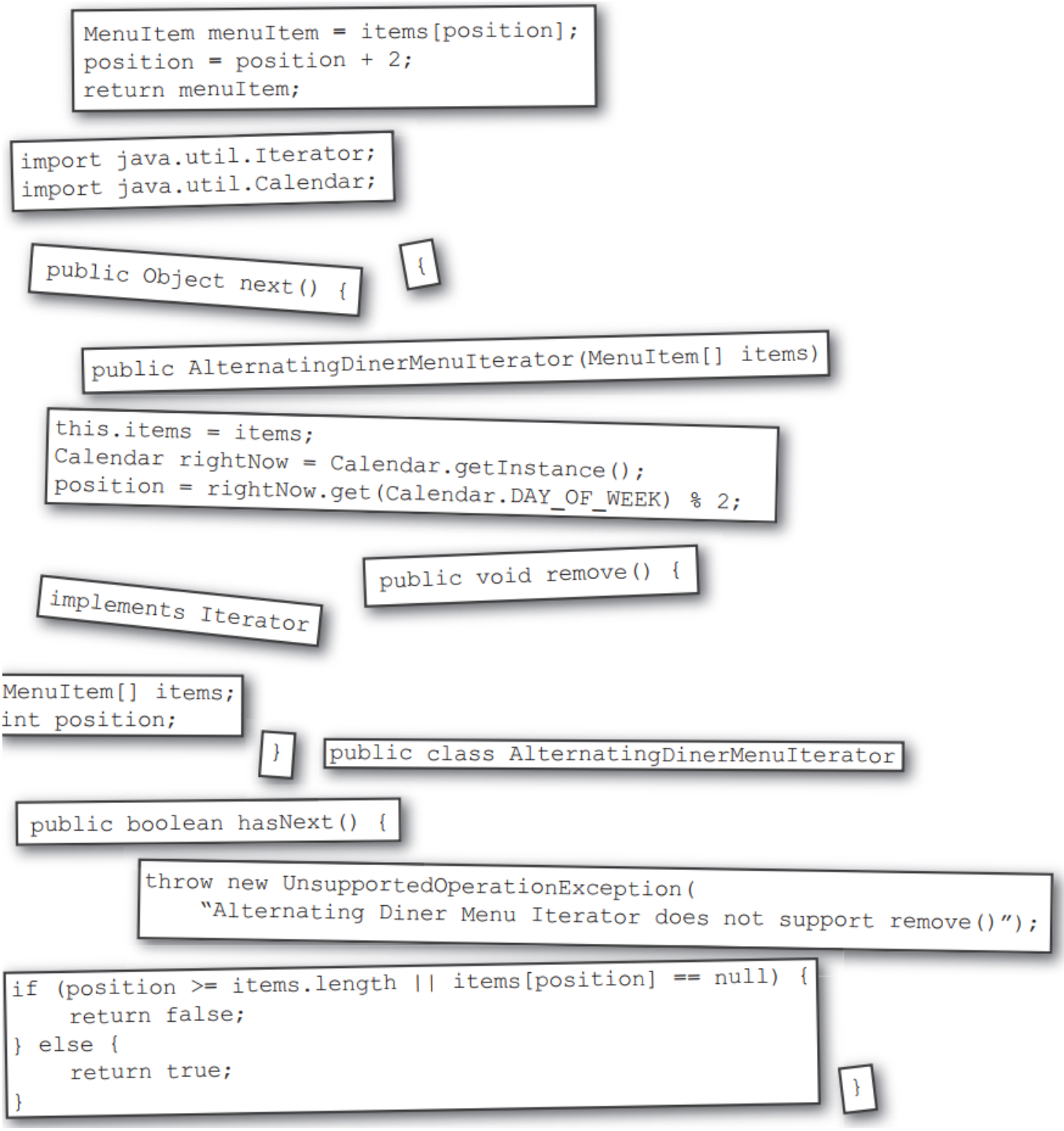
Iterate over the list and print each item.

Bạn cần sử dụng tính năng tổng quát mới của Java 5 để đảm bảo an toàn loại **for/in**. Hãy chắc chắn rằng bạn đã đọc chi tiết trước khi sử dụng **for/in**.

Code Magnets

Các đầu bếp đã quyết định rằng họ muốn có thể thay thế các món ăn trong thực đơn bữa trưa của họ; nói cách khác, họ sẽ cung cấp một số món vào Thứ Hai, Thứ Tư, Thứ Sáu và Chủ Nhật, và các món khác vào Thứ Ba, Thứ Năm và Thứ Bảy. Một người nào đó đã viết code cho một DinerMenu Iterator mới để nó

thay thế các món ăn trong menu, nhưng họ đã xáo trộn nó như một trò đùa. Một số đoạn code rơi xuống sàn và chúng quá nhỏ để nhặt lại, vì vậy hãy thoải mái thêm bao nhiêu trong số những thứ bạn cần.



Đáp án:



Waitress đã sẵn sàng cho thời gian chính?

Waitress đã đi được một chặng đường dài, nhưng bạn đã phải thừa nhận ba lần gọi đến **printMenu()** trông có vẻ "xấu xí".

Hãy xem thực tế, mỗi khi chúng ta thêm một menu mới, chúng ta sẽ phải mở triển khai Waitress và thêm code mới. Bạn có thể nói rằng "vi phạm nguyên tắc đóng mở" không?

Ba cuộc gọi **createdIterator()**.

```
public void printMenu() {
    Iterator pancakeIterator = pancakeHouseMenu.createIterator();
    Iterator dinerIterator = dinerMenu.createIterator();
    Iterator cafeIterator = cafeMenu.createIterator();

    System.out.println("MENU\n----\nBREAKFAST");
    printMenu(pancakeIterator);

    System.out.println("\nLUNCH");
    printMenu(dinerIterator);

    System.out.println("\nDINNER");
    printMenu(cafeIterator);
}
```

3 cuộc gọi đến **printMenu()**.

Mỗi khi chúng tôi thêm hoặc xóa một menu, chúng tôi sẽ phải mở code này để thay đổi.

Nó không phải là lỗi của Waitress. Chúng tôi đã thực hiện một công việc tuyệt vời là tách rời việc thực hiện menu và đưa phép lặp vào một iterator. Nhưng chúng tôi vẫn đang xử lý các menu với các đối tượng độc lập, riêng biệt – chúng tôi cần một cách để quản lý chúng cùng nhau.

Sử dụng sức mạnh bộ não

Waitress vẫn cần thực hiện ba lần gọi đến **printMenu()**, một lần gọi cho mỗi menu. Bạn có thể nghĩ ra cách kết hợp các menu để chỉ cần thực hiện một lần gọi không?



```
public class Waitress {
    ArrayList menus;

    public Waitress(ArrayList menus) {
        this.menus = menus;
    }

    public void printMenu() {
        Iterator menuIterator = menus.iterator();
        while(menuIterator.hasNext()) {
            Menu menu = (Menu)menuIterator.next();
            printMenu(menu.createIterator());
        }
    }

    void printMenu(Iterator iterator) {
        while (iterator.hasNext()) {
            MenuItem menuItem = (MenuItem)iterator.next();
            System.out.print(menuItem.getName() + ", ");
            System.out.print(menuItem.getPrice() + " -- ");
            System.out.println(menuItem.getDescription());
        }
    }
}
```

Bây giờ chúng ta chỉ cần lấy một **ArrayList** của các menu

Và chúng tôi duyệt qua các menu, chuyển từng iterator của menu sang phương thức **printMenu()** được overload.

Không có thay đổi ở đây

Điều này có vẻ khá tốt, mặc dù chúng tôi đã mất tên của các menu, nhưng chúng tôi có thể thêm tên vào mỗi menu.

(xem tiếp.. (https://toihocdesignpattern.com/chuong-4-factory-pattern-va-abstract-factory-pattern-phan-2.html)Iterator Pattern và Composite Pattern phần 2 (https://toihocdesignpattern.com/chuong-9-iterator-pattern-va-composite-pattern-p2-composite-pattern.html)).